

## СОДЕРЖАНИЕ

|                                                                                                                         |           |
|-------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....</b>                                                                                       | <b>2</b>  |
| <b>ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....</b>                                                                  | <b>4</b>  |
| <b>ВВЕДЕНИЕ.....</b>                                                                                                    | <b>5</b>  |
| <b>Глава 1. Теоретические основы устойчивости и безопасности<br/>распределённых информационных систем.....</b>          | <b>13</b> |
| 1.1. Проблемы устойчивости и защищённости распределённых систем при<br>высоконагруженном взаимодействии сервисов.....   | 13        |
| 1.2. Роль ограничения частоты запросов как механизма обеспечения<br>отказоустойчивости и предотвращения перегрузок..... | 19        |
| 1.3. Особенности обеспечения устойчивости и безопасности публичных<br>API в распределённых системах.....                | 24        |
| <b>Глава 2. Сравнительный анализ алгоритмов и архитектурных подходов к<br/>управлению нагрузкой.....</b>                | <b>29</b> |
| 2.1. Сравнительный анализ алгоритмов ограничения частоты запросов....                                                   | 29        |
| 2.2. Архитектурные модели интеграции механизмов ограничения запросов<br>в распределённые системы.....                   | 35        |
| 2.3. Компромиссы между точностью, отказоустойчивостью и<br>адаптивностью механизмов ограничения запросов.....           | 41        |
| <b>Глава 3. Экспериментальное исследование алгоритмов ограничения<br/>запросов в распределённой системе.....</b>        | <b>47</b> |
| 3.1. Методика проведения эксперимента и описание архитектуры<br>прототипа системы.....                                  | 47        |
| 3.2. Реализация и сравнение алгоритмов ограничения запросов в условиях<br>различных сценариев нагрузки.....             | 57        |
| <b>ЗАКЛЮЧЕНИЕ.....</b>                                                                                                  | <b>74</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....</b>                                                                            | <b>77</b> |
| <b>СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА.....</b>                                                                            | <b>84</b> |

## ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Адаптивное лимитирование – режим управления ограничениями частоты запросов, при котором параметры алгоритма изменяются в зависимости от наблюдаемого профиля нагрузки

Каскадный отказ – последовательная деградация взаимосвязанных компонентов распределённой системы вследствие локального сбоя или перегрузки

Легитимный трафик – поток запросов от добросовестных пользователей и интеграций, не направленный на нарушение доступности сервиса

Распределённый лимитер – механизм ограничения запросов, работающий в многосервисной или многоузловой среде с разделяемым состоянием

Сценарий нагрузки – формализованный профиль входящего трафика, используемый для воспроизводимого тестирования устойчивости системы

Устойчивость системы – способность информационной системы сохранять работоспособность при отклонениях нагрузки и частичных отказах компонентов

Переменная нагрузка – входящий поток запросов, интенсивность и структура которого изменяются во времени в пределах штатного или ожидаемого поведения системы.

Аномальная нагрузка – входящий поток запросов, параметры которого существенно отклоняются от ожидаемого профиля и создают риск перегрузки, деградации или отказа компонентов системы.

Штатная нагрузка – входящий поток запросов, соответствующий ожидаемому режиму работы системы и не приводящий к превышению проектных ограничений по пропускной способности, задержке и числу ошибок.

Пиковая нагрузка – временное увеличение интенсивности входящих запросов относительно обычного уровня, возникающее в результате

сезонного роста, маркетинговых кампаний, массовых уведомлений, релизов или иных штатных факторов.

Всплесковая нагрузка – кратковременное резкое увеличение интенсивности запросов, после которого поток возвращается к базовому уровню. Всплеск может быть как легитимным, так и аномальным в зависимости от причины и влияния на систему.

DDoS-нагрузка – аномальный поток запросов, формируемый множеством источников и направленный на истощение ресурсов системы, рост задержек, увеличение числа отказов и нарушение доступности сервиса.

Ретрай-шторм – ситуация, при которой клиенты или интеграции массово повторяют неуспешные запросы, из-за чего исходная перегрузка усиливается и может приводить к росту очередей, задержек и длительному окну нестабильности.

## **ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ**

API – Application Programming Interface, программный интерфейс приложения

CI95 – 95% доверительный интервал

DDoS – Distributed Denial of Service, распределённая атака отказа в обслуживании

HTTP – HyperText Transfer Protocol

KPI – Key Performance Indicator, ключевой показатель эффективности

ML – Machine Learning, машинное обучение

RPS – Requests Per Second, количество запросов в секунду

SLA – Service Level Agreement, соглашение об уровне сервиса

SLO – Service Level Objective, целевой показатель уровня сервиса

UX – User Experience, пользовательский опыт

## ВВЕДЕНИЕ

Актуальность исследования определяется тем, что цифровые сервисы стали критической инфраструктурой для повседневной жизни людей и бизнеса.

Недоступность программных интерфейсов приложений (API) сегодня означает не только технический сбой. Из-за неё могут остановиться платежи, нарушиться логистика и ухудшиться работа медицинских, образовательных и государственных сервисов.

По данным облачных операторов, число и интенсивность распределённых атак отказа в обслуживании (DDoS) продолжили расти в 2025 году, включая гипербольшим атакам в диапазонах Tbps и сотен миллионов запросов в секунду [1] [2].

В такой ситуации ограничение запросов нужно не только для ускорения системы. Его главная задача — сохранить доступность цифровых услуг для добросовестных пользователей даже при аномальной нагрузке.

Чтобы сравнивать варианты не на уровне ощущений, в работе используются SLA, SLO и 95% доверительный интервал (CI95). SLA и SLO задают требуемый уровень сервиса, а CI95 показывает разброс результатов между повторными прогонами.

Классические алгоритмы ограничения частоты запросов известны и широко применяются, но в реальных распределённых системах они сталкиваются с компромиссами между точностью, доступностью и задержками [3] [4] [5].

На практике механизм ограничения запросов не всегда принимает одинаковое решение на всех серверах в один и тот же момент. Если запросы идут через несколько серверов, сведения о текущей нагрузке доходят до них с задержкой. Поэтому ограничение работает не как один идеальный счётчик, а как механизм, который в целом удерживает поток запросов в допустимых пределах [3] [4].

Поэтому важно не только правильно написать сам алгоритм. Нужно заранее решить, какие алгоритмы сочетать, где проверять запросы и как менять поведение системы при изменении трафика.

Проблема особенно обостряется для сервисов с высокой социальной нагрузкой:

финтех, электронная коммерция, логистика, образование, государственные цифровые порталы.

В таких системах инцидент доступности затрагивает не только технические метрики, но и реальные действия пользователей: оплату, заказ, получение услуги, подтверждение операций.

Поэтому в работе используется прикладной критерий полезности: насколько решение позволяет сохранить доступ для добросовестных пользователей в период перегрузки.

С практической стороны выделяются три типовые ошибки эксплуатации:

1. Отсутствие лимитов до возникновения инцидента. В этом случае система продолжает принимать весь входящий поток запросов независимо от текущей пропускной способности сервисов. При резком росте нагрузки, ошибочной интеграции или DDoS-подобном сценарии это приводит к неконтролируемому заполнению очередей, росту задержек, увеличению числа повторных запросов и риску каскадной деградации. Иными словами, лимитирование начинает рассматриваться как реакция на уже произошедший сбой, а не как заранее встроенный механизм устойчивости.

2. Чрезмерно жёсткие статические лимиты, не адаптирующиеся к естественным пикам нагрузки. Такая политика может приводить к тому, что в периоды легитимного роста активности система ошибочно воспринимает легитимный трафик как избыточный и начинает отклонять запросы. В результате пользователи сталкиваются с отказами и задержками не из-за реальной атаки, а из-за недостаточной гибкости механизма лимитирования.

3. Несогласованность правил лимитирования между разными слоями системы. На практике ограничения могут одновременно применяться на уровне API-шлюза, отдельных сервисов, клиентских ключей, маршрутов и бизнес-операций. Если эти правила настраиваются независимо и не согласуются между собой, возникает риск конфликтующего поведения: один слой уже отклоняет запросы, другой допускает их, а третий применяет собственные пороги и коды ошибок. Это затрудняет диагностику, делает поведение API непредсказуемым для клиентов и снижает управляемость системы в период перегрузки.

Предлагаемый подход закрывает эти ошибки за счёт комбинированной политики. В обычном режиме система старается сохранить доступность, а при аномальной нагрузке переходит в более строгий защитный режим.

#### **Степень разработанности темы.**

Проблематика ограничения частоты запросов в распределённых системах является достаточно разработанной в научной и инженерной литературе: описаны базовые алгоритмы (Fixed Window, Sliding Window, Token Bucket, Leaky Bucket), а также подходы к их реализации в платформенных решениях и API-шлюзах [3] [4] [5].

При этом в опубликованных материалах преобладают либо теоретические сопоставления алгоритмов, либо рекомендации по эксплуатации, ориентированные на конкретные платформы.

В существенно меньшей степени представлены воспроизводимые сравнительные исследования, в которых единообразно оцениваются скорость, доступность легитимного трафика и устойчивость в аномальных сценариях на одном стенде.

#### **Решаемая проблема.**

Даже при наличии готовых алгоритмов остаётся практический вопрос. Нужно понять, какой режим выбрать и как его настроить, если нагрузка постоянно меняется.

Статический лимит часто работает плохо в одном из двух случаев. В обычном пике он может быть слишком жёстким, а при атаке — слишком мягким [4].

Работа решает задачу построения воспроизводимого подхода к адаптивному управлению ограничениями частоты запросов в секунду (RPS), при котором баланс между доступностью, защитой и задержкой задаётся измеримыми метриками.

Объект исследования: распределённые информационные системы с публичными и внутренними API.

Предмет исследования: алгоритмы и архитектуры ограничения частоты запросов в условиях переменной и аномальной нагрузки.

Цель работы состоит в разработке и экспериментальной проверке адаптивного подхода к ограничению частоты запросов. Такой подход должен повышать устойчивость и защищённость распределённой системы по сравнению со статическими лимитами.

Для достижения цели поставлены следующие задачи:

1. Рассмотреть теоретические основы устойчивости и защищённости распределённых информационных систем в условиях высоконагруженного взаимодействия сервисов.
2. Проанализировать роль ограничения частоты запросов как механизма предотвращения перегрузок, повышения отказоустойчивости и защиты публичных API от избыточной или аномальной нагрузки.
3. Выявить особенности применения механизмов ограничения запросов в публичных API распределённых систем, включая требования к доступности, безопасности, масштабируемости и качеству пользовательского опыта.
4. Провести сравнительный анализ основных алгоритмов ограничения частоты запросов, таких как Fixed Window, Sliding Window, Token Bucket и Leaky Bucket, с точки зрения точности, простоты реализации, устойчивости к всплескам нагрузки и применимости в распределённой среде.



5. Рассмотреть архитектурные модели интеграции механизмов ограничения запросов в распределённые системы и определить их влияние на производительность, отказоустойчивость и управляемость системы.

6. Выявить ключевые компромиссы между точностью, отказоустойчивостью и адаптивностью механизмов ограничения запросов при их применении в распределённых информационных системах.

7. Разработать прототип распределённой системы для экспериментального исследования алгоритмов ограничения запросов, включающий набор сервисов, механизм генерации нагрузки и средства сбора метрик.

8. Провести экспериментальное сравнение алгоритмов ограничения запросов в условиях различных сценариев нагрузки, включая равномерную, переменную, пиковую и аномальную нагрузку.

9. Оценить влияние различных алгоритмов ограничения запросов на устойчивость системы и качество обслуживания пользователей по таким показателям, как доля отклонённых запросов, стабильность обработки нагрузки, задержки ответа и корректность соблюдения заданных лимитов.

10. Сформулировать практические выводы и рекомендации по выбору алгоритмов и архитектурных подходов к ограничению частоты запросов в распределённых системах.

Методы исследования: системный анализ, архитектурное моделирование, сравнительный эксперимент, статистический анализ метрик, инженерная интерпретация результатов.

Методологическую и теоретическую основу исследования составляют принципы проектирования отказоустойчивых распределённых систем, подходы к анализу угроз информационной безопасности, методы сценарного нагрузочного тестирования, методы мониторинга временных рядов и практики SRE по управлению перегрузками.

Практическая значимость работы состоит в том, что предложенный подход ориентирован на эксплуатацию в реальных командах:

1. Позволяет уменьшать вероятность массовой недоступности пользовательских сервисов.
2. Снижает риск ресурсного истощения и «лавины ретраев» в микросервисах.
3. Даёт воспроизводимую методику выбора алгоритма под конкретный профиль нагрузки.
4. Поддерживает поэтапное внедрение без переписывания всей платформы.

Научно-практическая новизна работы заключается не в создании нового алгоритма лимитирования, а в проверке гипотезы - возможно ли на реальном стенде воспроизвести работу по переключению режимов ограничения запросов в зависимости от профиля нагрузки и метрик сервисов на лету.

#### **Положения, выносимые на защиту.**

1. Для распределённых API статическая политика RPS-лимитирования недостаточна в условиях смешанной нагрузки, поскольку один и тот же фиксированный режим не обеспечивает одновременно высокую доступность легитимного трафика и достаточную жёсткость фильтрации аномального потока.
2. Выбор алгоритма лимитирования должен зависеть от профиля нагрузки и метрик сервисов.
3. Разделение режимов лимитирования на базовый и защитный позволяет снизить противоречие между пользовательской доступностью и защитой от перегрузки по сравнению с применением единого статического алгоритма.
4. Адаптивное переключение между Token Bucket и Sliding Window на основе телеметрии, краткосрочного и долгосрочного прогноза повышает среднюю доступность системы в смешанном сценарии нагрузки по сравнению со статическими режимами.
5. Безопасное применение адаптивного лимитирования требует ограничений на переключение режимов: минимального времени удержания,

контроля восстановления, предельных значений параметров и возможности отката. Без таких ограничений адаптивный контур может сам стать источником нестабильности или чрезмерно долгого сохранения жёсткого режима.

Степень достоверности результатов обеспечивается воспроизводимым экспериментальным стендом, повторяемыми тестовыми сценариями, фиксацией исходных CSV-артефактов, использованием CI95 для статической матрицы и сопоставлением результатов по нескольким независимым метрикам: успешность, доля HTTP ошибок 429, p95/p99-задержка (перцентильные показатели времени отклика), число переключений и состояние инфраструктурных компонентов.

Апробация результатов выполнена в проекте RPS-limiter (<https://github.com/Islam-Y/RPS-limiter>), разработанном для проверки подходов к ограничению частоты запросов. Проект представляет собой тестовый стенд, в котором можно воспроизводимо подавать нагрузку на сервис, включать разные алгоритмы лимитирования, фиксировать количество пропущенных и отклонённых запросов, измерять задержку и наблюдать поведение системы при перегрузке.

В рамках апробации были реализованы основные компоненты стенда: генератор нагрузки, сервис лимитирования, целевой сервис обработки запросов, Redis для хранения состояния лимитера, средства мониторинга Prometheus/Grafana, скрипты для повторяемых нагрузочных экспериментов и инструкция для локального запуска.

Личный вклад автора состоит в постановке и исследовании задачи адаптивного управления нагрузкой в распределённых API на основе ограничения частоты запросов. Автором выполнено теоретическое обоснование роли RPS-лимитирования как механизма устойчивости и защищённости, проведён сравнительный анализ алгоритмов Fixed Window, Sliding Window и Token Bucket, определены архитектурные условия их применения, сценарии нагрузки, метрики и критерии эффективности. Для

проверки предложенного подхода реализован экспериментальный прототип RPS-limiter, использованный как средство воспроизводимой оценки. На основе проведённых экспериментов автором обосновано разделение базового и защитного режимов лимитирования, показана применимость адаптивного переключения между Token Bucket и Sliding Window и сформулированы практические рекомендации по безопасному внедрению адаптивного лимитирования в распределённых системах.

Структура работы организована от теоретического обоснования проблемы устойчивости и безопасности распределённых систем — к сравнительному анализу алгоритмов и архитектурных подходов лимитирования, а затем к экспериментальной проверке адаптивного подхода. Работа состоит из введения, трёх глав, заключения, списка использованных источников и списка иллюстративного материала.

# Глава 1. Теоретические основы устойчивости и безопасности распределённых информационных систем

## 1.1. Проблемы устойчивости и защищённости распределённых систем при высоконагруженном взаимодействии сервисов

В распределённой архитектуре отказ одного элемента редко остаётся локальным событием.

Если часть кластера теряет доступность, трафик перераспределяется на оставшиеся узлы, и при недостаточном запасе ёмкости система переходит в режим каскадной деградации [7].

Можно сделать вывод, что устойчивость распределённой системы должна проектироваться не только для усреднённого показателя уровня нагрузки, но и для отклоняющихся режимов, в которых одновременно возрастают интенсивность входящего трафика, задержка обработки запросов и количество повторных обращений.

Типовой сценарий каскадной деградации в распределённой системе показан на рисунке 1.

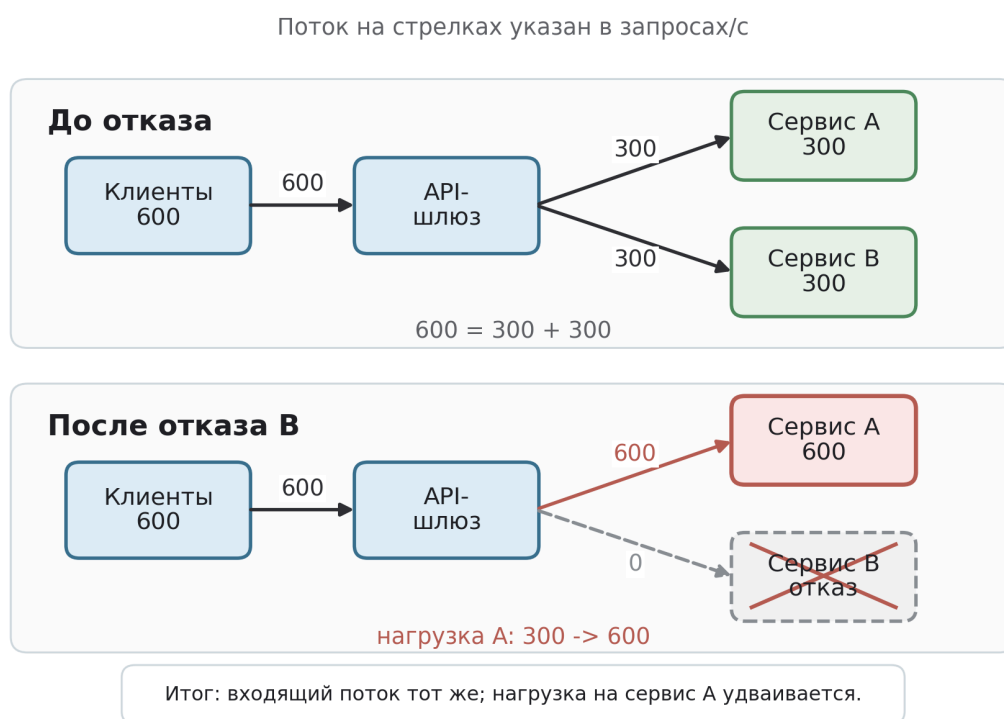


Рисунок 1 – Перераспределение нагрузки при отказе экземпляра сервиса

Каскадные отказы часто развиваются как самоподдерживающийся процесс: замедление одного компонента приводит к росту очередей, таймаутам и повторным запросам со стороны клиентов или соседних сервисов. Эти повторные запросы создают дополнительную нагрузку, из-за чего задержка увеличивается ещё сильнее. В результате локальная перегрузка может распространиться на другие компоненты и привести к деградации всей системы. [7].

При этом ресурсы заканчиваются не только по CPU, но и по потокам, соединениям, памяти и сетевым буферам.

Даже если система потом возвращается в норму, окно деградации может быть достаточно долгим, чтобы пользователи массово потеряли доступ к сервису, а SLA был нарушен на протяжении целого операционного интервала.

Для прикладного анализа удобно разделять источники перегрузки на три группы:

1. Внешний злонамеренный трафик (боты, DDoS, сканирование API).
2. Внешний легитимный пик (маркетинговая кампания, релиз, сезонный всплеск).
3. Внутренне индуцированная перегрузка (ретрай-шторм, ошибочная интеграция, клиент или интеграция часто отправляющие повторные запросы к API).

Такая классификация обоснована тем, что разные группы требуют разных режимов лимитирования.

Например, при легитимном пике приоритетом является сохранение доступности для максимальной доли добросовестных запросов, а при атаке приоритет смещается в сторону жёсткой фильтрации.

Стратегия, предполагающая наличие лишь одного способа лимитирования на все случаи в такой постановке может уступать адаптивной.

Отдельную категорию риска формируют DDoS и ботнет-кампании.

Публичные отчёты Cloudflare за 2025 год фиксируют резкий рост числа атак, в том числе гипербольшим, и подтверждают, что атаки становятся не только масштабнее, но и более автоматизированными [1] [2].

Для продуктовых сервисов это означает, что безопасность должна быть постоянной и неотъемлемой частью архитектуры, а не временной надстройкой на период инцидента.

Показательным примером подобных угроз являются атаки с усилением трафика через серверы memcached. Memcached — это система кэширования данных, которая при неправильной сетевой конфигурации может быть доступна из интернета. В таком случае злоумышленник отправляет на такой сервер небольшой запрос с подменённым адресом отправителя, а сервер направляет значительно больший ответ уже на адрес жертвы. В результате небольшой объём исходящего трафика со стороны атакующего превращается в многократно больший поток, направленный на атакуемый сервис.

Этот вектор демонстрирует, как небольшие входные пакеты могут формировать несоразмерно большой отражённый трафик; фактор усиления может быть очень высоким [8].

Уязвимость класса uncontrolled resource consumption для такого сценария зафиксирована и в NVD (CVE-2018-1000115) [9].

Практический вывод для публичных API состоит в том, что защита должна ограничивать не только «объём в секунду», но и скорость потребления критичных операций на уровне ключа клиента, маршрута и метода.

Кроме атак, существенную долю инцидентов формируют «легитимные» всплески: маркетинговые акции, релизы, массовые push-уведомления, неудачные клиентские ретраи.

С точки зрения пользователя причины не важны: результат один и тот же, если API недоступен.

Качество решения в такой постановке оценивается через показатели удовлетворённости пользователей: сохраняется ли доступность сервиса для добросовестного клиента в пике и насколько предсказуемо происходит деградация.

На практике полезно оценивать не только факт инцидента, но и его человеческую стоимость:

1. Потеря доступа к базовым действиям (авторизация, платёж, обращение в поддержку).
2. Потеря доверия к сервису из-за непредсказуемых отказов.
3. Вторичный ущерб от «цепочки зависимостей», когда недоступность одного API блокирует несколько пользовательских сценариев сразу.

Это согласуется с современными стандартами и практиками управления рисками [10] [11].

При недоступности API пользователь теряет услугу; значит, отказоустойчивость является не вспомогательной, а базовой частью безопасности сервисов.

Дополнительно следует учитывать эксплуатационный аспект: даже кратковременная деградация может вызвать лавину повторов со стороны мобильных клиентов и внешних интеграторов.

Поэтому архитектура защиты должна предполагать корректное поведение клиента в условиях ограничения, включая HTTP коды 429, задержки повторов и прогнозируемое восстановление квот [12].

Без этого система теряет управляемость: перегрузка возвращается повторно уже после первичного пика.

Сводная связь угроз и эффекта для пользователей представлена в таблице 1.

Таблица 1 – Источники перегрузки и прикладные последствия.

| Источник | Типичный механизм деградации | Риск для пользователя | Приоритетный контрмеханизм |
|----------|------------------------------|-----------------------|----------------------------|
|          |                              |                       |                            |



|                 |                                                  |                                |                                                        |
|-----------------|--------------------------------------------------|--------------------------------|--------------------------------------------------------|
| DDoS/бот-трафик | Резкий рост входящего RPS, истощение ресурсов    | Недоступность API, рост ошибок | Жёсткое лимитирование, фильтрация по ключам/маршрутам  |
| Легитимный пик  | Очереди и рост задержки в пиковые интервалы      | Замедление операций, таймауты  | Мягкий допуск с контролем всплесков, адаптация порогов |
| Retry-шторм     | Повтор ошибок усиливает поток повторных запросов | Длительное окно нестабильности | 429 + Retry-After + ограничение частоты повторов       |

Для проектирования устойчивости важно формализовать целевые KPI, а не ограничиваться общим тезисом «сервис должен держать нагрузку».

Для практического применения важно контролировать:

1. Долю успешных ответов в штатном профиле (целевой порог, например, не ниже 95%).
2. Долю отказов с кодом 5xx на защищаемом сервисе в аномальном профиле.
3. Время восстановления после пика (в секундах до возврата к штатной задержки).
4. Долю ложных отклонений, то есть код 429 для трафика, который не является вредоносным.

При отсутствии указанных показателей невозможно обосновать, что выбранная стратегия обеспечивает требуемый эффект для пользователей и бизнеса.

При этом устойчивость должна оцениваться в горизонте полного инцидента, а не в одном коротком срезе.

Сервис может демонстрировать приемлемое состояние в начале пика, но деградировать через несколько минут из-за накопления очередей и повторов.

Поэтому в методике эксперимента (глава 3) акцент сделан на повторные прогоны и анализ разброса, чтобы не принимать решение по случайному удачному прогону.

### **Вывод по разделу 1.1.**

Подход, который одновременно учитывает перегрузки, атакующий трафик и поведение легитимных клиентов, эффективнее узконаправленной защиты «только от DDoS».

Он обеспечивает более хороший уровень удовлетворённости пользователя: не только блокирование избыточного трафика, но и сохранение работы системы в пределах контролируемого качества.

## **1.2. Роль ограничения частоты запросов как механизма обеспечения отказоустойчивости и предотвращения перегрузок**

Ограничение частоты запросов выполняет роль контроллера.

Когда входящий поток превышает проектные параметры системы, лимитер начинает отклонять часть запросов, предотвращая каскадное распространение ошибок на другие сервисы [13].

Такой режим предпочтительнее полной недоступности, поскольку сохраняет частичную работоспособность для значимой доли пользователей.

Лимитирование создаёт управляемое обратное давление (защита от каскадирования) на уровне API.

Система не должна принимать неограниченный поток и обрабатывать его без ограничения; корректная стратегия предполагает ограничение входа до уровня, совместимого с текущей пропускной способностью.

Именно это предотвращает перегрузку сервисов, создавая управляемый поток деградации.

В распределённых сервисах такой режим часто является единственным способом удерживать ядро бизнес-операций в рабочем состоянии.

На уровне HTTP общепринятой реакцией на такое поведение является код 429 Too Many Requests, формализованный в RFC 6585 [12].

Рекомендация возвращать заголовок «Повторите позже» указывает клиенту, сколько времени следует подождать перед повторным запросом. Благодаря этому клиент не начинает сразу многократно повторять неуспешную операцию, а делает паузу, снижая дополнительную нагрузку на уже перегруженный сервис. [12].

Это превращает лимитирование из простого отказа системы в управляемый протокол взаимодействия клиента и сервера.

Важно, что код 429 является не «ошибкой приложения», а частью контрактной политики API.

Клиент, который корректно обрабатывает код ошибки 429 и принимает во внимание заголовок «Повторите позже», улучшает устойчивость всей системы.

Клиент, который игнорирует эти сигналы и продолжает агрессивные повторы, становится источником вторичной перегрузки.

Полезность лимитирования усиливается в связке с требованиями к клиентским SDK и интеграционным регламентам.

В управляемых API-платформах лимитирование закреплено как штатный механизм.

Например, AWS API Gateway использует Token Bucket и отдельно отмечает, что лимит может срабатывать с небольшими отклонениями, а не как абсолютно точный счётчик для каждого запроса [3] [14].

Azure API Management также подчёркивает, что в распределённой архитектуре точность лимита не может быть абсолютно строгой в каждый момент времени [4].

Эта оговорка критична для корректной инженерной интерпретации результатов.

В контексте эксплуатации это означает следующее:

1. Лимит задаёт рабочий диапазон, а не математически мгновенную гарантию.
2. Краткие отклонения допустимы, если не приводят к системной деградации.
3. Решение оценивается по интегральному эффекту (доступность, задержка и доля ошибок 429), а не по единичному «проскочившему» запросу.

Практика крупных API-провайдеров показывает, что лимиты нужны не только для безопасности, но и для справедливого распределения ресурсов между клиентами.

Cloudflare документирует глобальные API-лимиты и возврат кода 429 при превышении [15].

GitHub публикует разные квоты для аутентифицированных и неаутентифицированных запросов, тем самым выравнивая нагрузку и снижая риск злоупотреблений [16].

Это важно не только для владельца API. Лимитирование помогает сохранить доступ для добросовестных клиентов, когда часть трафика создаёт перегрузку.

Если один агрессивный клиент без ограничений занимает большую часть ресурса, остальные пользователи фактически лишаются услуги.

Механизм квот и ограничений восстанавливает ожидаемый от сервера доступ, что особенно важно для публичных и партнёрских API.

Для конечных пользователей это выражается в управляемом режиме деградации сервиса вместо полного отказа.

Для операторов это даёт предсказуемость: видны лимиты, понятны окна, можно анализировать долю отклонений как нормальный операционный сигнал.

Для бизнеса это уменьшает вероятность каскадного простоя и неконтролируемого роста инфраструктурных затрат.

В системах, где ресурсы делят несколько клиентов, ограничение частоты запросов также выступает инструментом экономической защиты.

Оно предотвращает ситуацию, когда одна интеграция или один клиент резко увеличивает потребление и вынуждает масштабировать всю платформу в аварийном режиме.

Механизм ограничения запросов помогает не только защитить сервис от перегрузки, но и контролировать расходы: если лимиты настроены плохо, даже легитимный трафик может создать лишнюю нагрузку и увеличить стоимость эксплуатации.

Лимитирование не является единственным инструментом и не заменяет WAF, мониторинг, авторизацию или кэширование.

Его сила в том, что он работает как первый рубеж управления скоростью потребления ресурсов.

Именно этот рубеж предотвращает переход от локального всплеска к системному инциденту.

На практике оправдана многоуровневая политика:

1. Глобальный лимит на уровне API-шлюза (грубая отсечка).
2. Лимит по клиенту/ключу (балансировка между потребителями).
3. Лимит по конечной точке или категории операций (защита критичных маршрутов).
4. Более строгий профиль при признаках атаки и более мягкий профиль в штатном режиме.

Прикладной эффект лимитирования для разных участников приведён в таблице 2.

Таблица 2 – Прикладной эффект лимитирования для разных участников.

| <b>Участник</b>       | <b>Что получает без лимитирования</b>     | <b>Что получает при корректном лимитировании</b>                       |
|-----------------------|-------------------------------------------|------------------------------------------------------------------------|
| Конечный пользователь | Риск полного отказа сервиса в пике        | Сохранение частичной доступности и предсказуемое поведение API         |
| Оператор платформы    | Непредсказуемый рост инцидентов и алертов | Управляемые метрики - доля 429/задержки и понятные режимы реагирования |
| Бизнес-заказчик       | Простои и перерасход инфраструктуры       | Контролируемый уровень качества и затрат                               |

Отдельно следует учитывать, что лимитирование требует калибровки, иначе оно превращается в источник ложных отказов.

Практический процесс калибровки включает:

1. Сбор базовой телеметрии по маршрутам в нормальном режиме.
2. Выбор стартовых лимитов с запасом относительно медианного трафика.

3. Контроль p95/p99-задержка и доли 429 ошибок после включения ограничений.

4. Итеративную корректировку порогов по результатам наблюдения.

Такой подход предпочтительнее «разового выбора» лимита, поскольку учитывает реальное распределение нагрузки, а не экспертную оценку без эмпирической верификации.

Для публичных API также важно документировать политику лимитов для внешних интеграторов.

Наличие прозрачных квот и ожидаемого поведения при превышении снижает количество инцидентов, вызванных некорректной клиентской логикой.

Лимитирование здесь выступает одновременно техническим и договорным механизмом устойчивости.

### **Вывод по разделу 1.2.**

По сравнению с политикой «обслуживать весь входящий поток без ограничений», управляемое ограничение запросов эффективнее сохраняет полезную производительность.

Это повышает отзывчивость сервиса, то есть в критической ситуации больше пользователей получают ответ, пусть и при частичных ограничениях.

В связке со сценарно-зависимыми настройками (штатный/аномальный режим) такой подход даёт лучший баланс между доступностью и защитой, чем единый фиксированный порог.

### **1.3. Особенности обеспечения устойчивости и безопасности публичных API в распределённых системах**

Публичный API упрощает подключение внешних клиентов, но одновременно открывает сервис для неизвестного трафика

Чем проще внешнему клиенту подключиться, тем выше требования к контролю потребления ресурсов и авторизации.

Поэтому при проектировании API важно одновременно обеспечить удобство подключения для внешних клиентов и заранее определить допустимые пределы использования: кто может обращаться к API, какие операции доступны, как часто можно отправлять запросы и что происходит при превышении лимитов.

В отличие от внутренних сервисных интерфейсов, публичный API работает в условиях неполного доверия к клиенту.

Провайдер не контролирует ни сетевой контур клиента, ни его логику повторных запросов, ни частоту вызовов.

Устойчивость публичного API должна быть встроенной характеристикой: политика допуска, лимиты, валидация входа и наблюдаемость должны задаваться уже на этапе проектирования контракта.

Фундаментом предсказуемости интерфейса выступает стандартизованное описание API.

OpenAPI задаёт язык, на котором команда и клиенты одинаково понимают контракты, форматы и ошибки [17].

Чем прозрачнее контракт, тем меньше «шумных» запросов из-за неверной интеграции и тем ниже вероятность случайной перегрузки.

Практический эффект стандартизованного контракта состоит в снижении непреднамеренного шумового трафика:

1. Клиенты корректнее формируют запросы и обрабатывают ошибки.
2. Уменьшается число лишних повторов из-за неправильной трактовки ответов.



3. Снижается доля случайных пиков, вызванных дефектами интеграции.

Качество контракта напрямую связано с качеством нагрузочного профиля API.

На уровне идентификации и делегирования доступа индустриальным стандартом остаётся OAuth 2.0 [18].

В контексте устойчивости это важно, потому что корректная аутентификация и модель scope-ов позволяют ограничивать не только скорость, но и объём доступных операций для каждого класса клиентов.

Иными словами, безопасность и отказоустойчивость здесь взаимосвязаны: чем точнее модель прав, тем точнее можно настроить лимиты под разные роли и сценарии.

Например, системная интеграция с бизнес-критичными операциями может иметь отдельный профиль квот, отличный от публичного developer-token.

Это снижает риск, что «общий» лимит случайно заблокирует критичный канал обслуживания.

В OWASP API Security Top 10 (2023) отдельно отмечен риск ситуации, когда API позволяет клиенту расходовать слишком много ресурсов без достаточных ограничений [19].

Механизм ограничения запросов для публичного API должен рассматриваться как обязательный элемент базовой модели угроз, а не как опциональное улучшение.

Для современных API недостаточно ограничиться только общим лимитом по IP.

Минимально необходимый уровень обычно включает несколько осей:

1. Лимиты на токен/клиента.
2. Лимиты на маршрут и метод.
3. Лимиты на класс операций (чтение/изменение/тяжёлые вычисления).
4. Отдельные защитные пороги для подозрительных паттернов.

Такой набор ограничений лучше защищает API не только от намеренных атак и злоупотреблений, но и от ошибок добросовестных клиентов, например слишком частых повторных запросов, неправильно настроенных интеграций или неудачной логики опроса сервиса.

Нагрузка на публичный API чаще всего меняется неравномерно. В один и тот же момент могут приходить обычные пользовательские запросы, всплески активности, автоматические парсеры, фоновые интеграции и повторные запросы после ошибок. Из-за этого один заранее заданный лимит часто работает плохо: если он слишком высокий, сервис может перегрузиться; если слишком низкий, будут страдать обычные пользователи. Поэтому нужен адаптивный слой, который меняет настройки ограничения запросов в зависимости от текущей ситуации.

Дополнительная специфика публичных API состоит в договорном характере взаимодействия.

Если политика лимитов непрозрачна или меняется резко, интеграторы получают нестабильную среду и начинают закладывать «агрессивные» защитные механизмы с частыми ретраями. Это усугубляет нагрузочный шум. Поэтому политика лимитов должна быть не только технически корректной, но и объяснимой: с публичными квотами, понятными кодами ответов и прогнозируемым поведением в пике [12] [15] [16].

#### **Дополнительный инженерный фактор — распределённость.**

Даже при корректной логике в одном узле глобальная картина в кластере может расходиться из-за задержек синхронизации.

Поэтому политика устойчивости должна учитывать, что идеальной синхронности в крупных системах нет, и строить защиту вокруг наблюдаемых метрик, а не только вокруг статических порогов.

Мониторинг становится частью безопасности продукта, а не только средством пост-фактум анализа.

Операционно значимыми являются:

1. Доля 429 ошибок по маршрутам и клиентам.

2. Задержка до и после лимитера.
3. Доля ошибок 5xx на защищаемом сервисе.
4. Динамика отклонений в пике и после восстановления.

Наличие таких метрик позволяет корректировать политику не интуитивно, а на основании наблюдаемого эффекта.

Для публичного API дополнительно критичен жизненный цикл изменений.

Если новые версии конечных точек или новые планы тарификации вводятся без пересмотра лимитов, распределение нагрузки резко меняется и защитный контур теряет адекватность.

Поэтому корректная эксплуатационная практика включает регламент:

1. Изменение контракта API сопровождается ревизией лимитов.
2. Изменение лимитов сопровождается обновлением документации для клиентов.
3. Для критичных маршрутов применяются канареечные включения новой политики.

Эта последовательность снижает риск, что технически корректное изменение вызовет массовые клиентские ошибки из-за непредсказуемого поведения ограничений.

С точки зрения пользы для пользователей такая практика даёт важный эффект: даже при развитии API сервис остаётся предсказуемым.

Пользователь не видит «скачкообразного» ухудшения качества только потому, что в платформе поменялись внутренние правила.

### **Вывод по разделу 1.3.**

Для публичных API более эффективен многоуровневый подход: контрактная прозрачность + авторизация + лимитирование + мониторинг.

В этой связке адаптивное управление лимитами даёт преимущество перед статическим алгоритмом, потому что снижает ложные блокировки легитимного трафика при сохранении защитной функции.

Для пользователя это означает более стабильный доступ к сервису при штатной работе сервиса и более предсказуемое поведение API в моменты аномальной нагрузки.

## Глава 2. Сравнительный анализ алгоритмов и архитектурных подходов к управлению нагрузкой

### 2.1. Сравнительный анализ алгоритмов ограничения частоты запросов

В работе рассматриваются четыре общепринятые группы алгоритмов: Fixed Window, Sliding Window, Token Bucket и Leaky Bucket.

Несмотря на схожую цель, они различаются по поведению в пике, требовательности к состоянию и стоимости для реализации в распределённой среде.

Для корректного сравнения алгоритмов важно разделять три уровня оценки:

1. Логическая корректность (правильно ли алгоритм описывает политику лимита).
2. Стоимость эксплуатации (память, CPU, сетевые обращения к хранилищу).
3. Пользовательский эффект (как часто легитимный запрос отклоняется и как меняется задержка).

Без такого разделения выбор алгоритма часто делается по «удобству реализации», а не по соответствию реальным свойствам целевого сервиса.

#### **Fixed Window.**

Плюсы: простота, минимальная стоимость вычислений, небольшие издержки эксплуатации.

Минусы: «краевой эффект» окна, из-за которого вблизи границы интервалов фактическая нагрузка может кратковременно превышать ожидаемую.

Этот алгоритм полезен там, где нужен базовый грубый контроль и минимальный накладной расход.

Формально для Fixed Window ограничение можно записать как  $\text{count}(\text{key}, t..t+T) \leq N$ .

При этом вблизи смены окна клиент может отправить почти  $2N$  запросов за короткий интервал вокруг границы.

Именно поэтому этот алгоритм подходит для грубого контроля, но плохо подходит для защиты очень «чувствительных» нижестоящих компонентов.

### **Sliding Window.**

Плюсы: более равномерный контроль частоты, меньшая уязвимость к краевым всплескам.

Минусы: выше стоимость эксплуатации, особенно в варианте с логом запросов; сложнее масштабировать в кластере.

На платформах вроде Azure API Management как раз используется идея скользящего окна для лимита по ключу [4].

Практически Sliding Window представлен двумя подвариантами:

1. Sliding log (хранение временных меток запросов) — выше точность, но выше стоимость памяти.
2. Sliding counter (аппроксимация по соседним окнам) — ниже стоимость, но появляется небольшая погрешность оценки.

### **Token Bucket.**

Плюсы: хорошее сочетание контроля средней скорости и допустимых коротких всплесков.

Минус подхода в том, что нужно аккуратно настроить скорость пополнения и допустимый всплеск. В распределённой схеме также нужна корректная синхронизация состояния.

Этот подход закреплён в AWS API Gateway и Spring Cloud Gateway как стандарт для API-нагрузки [3] [14] [20].

Для Token Bucket используется модель  $\text{tokens}(t) = \min(C, \text{tokens}(t_0) + r * dt)$ .

Запрос пропускается, если доступно достаточно токенов; иначе отклоняется.

Ключевое преимущество для пользовательского API состоит в том, что короткий всплеск может быть обработан без лавинообразного роста HTTP ошибок 429.

А риск заключается в неверной настройке **C** и **r**: слишком маленькие значения создают ложные блокировки, слишком большие ослабляют защитную функцию.

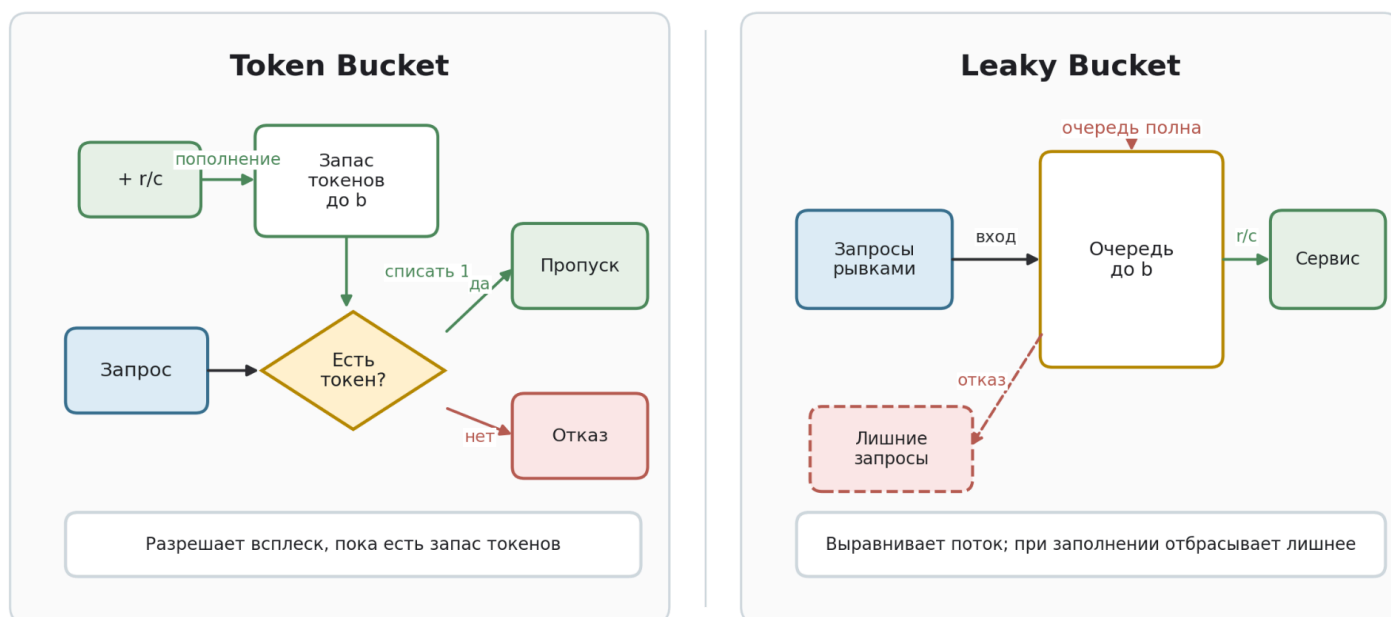
### **Leaky Bucket.**

Плюсы: алгоритм выравнивает поток запросов и защищает нижестоящие сервисы от резких всплесков нагрузки.

Минусы: часть запросов может ожидать обработки, из-за чего растёт задержка ответа. Это особенно важно для пользовательских сценариев, где человек ожидает быстрый ответ на короткий запрос.

Похожая логика используется в механизме ограничения запросов веб-сервера NGINX: он задаёт базовую скорость обработки и отдельно описывает, как поступать с кратковременным превышением лимита [21]. Такой подход удобен, когда нужно защитить слабый сервис ниже по цепочке, но его параметры нужно подбирать с учётом ожидаемой скорости ответа для пользователя.

Концептуальное сопоставление Token Bucket и Leaky Bucket приведено на рисунке 2.



Главное отличие: Token Bucket пропускает накопленный всплеск; Leaky Bucket выпускает запросы ровным темпом.

Рисунок 2 – Сравнение подходов Token Bucket и Leaky Bucket

Стартовое сопоставление по прикладному профилю приведено в таблице 3.

Таблица 3 – Выбор алгоритма по прикладному профилю.

| Прикладной сценарий                                    | Приоритет                                                                                                                                                | Наиболее подходящий алгоритм |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| Публичный API с переменным легитимным трафиком         | Нужно пропускать короткие всплески обычной активности, но не давать потоку расти бесконтрольно.                                                          | Token Bucket                 |
| Критичная точка API с риском скачкообразной перегрузки | Важно точнее контролировать число запросов в ближайший промежуток времени, чтобы короткий всплеск не перегрузил сервис.                                  | Sliding Window               |
| Простой внутренний сервис с ограниченным бюджетом      | Важны простота настройки, понятное поведение и минимальные затраты на сопровождение. Высокая точность ограничения здесь не является главным требованием. | Fixed Window                 |
| Узкий нижестоящий сервис с дорогими операциями         | Нужно не столько быстро отвечать на каждый запрос, сколько ровно выпускать                                                                               | Leaky Bucket                 |



|  |                                                       |  |
|--|-------------------------------------------------------|--|
|  | поток к слабому или дорогому сервису ниже по цепочке. |  |
|--|-------------------------------------------------------|--|

Таблица 3 в данном случае позволяет предварительно выбрать алгоритм.

В реальных проектах выбор уточняется по метрикам конкретного сервиса: доля ответов 429 в штатном режиме, p95/p99-задержка, количество ошибок и доля пропущенного аномального трафика.

С точки зрения алгоритмической стоимости различия также существенны:

1. Fixed Window обычно имеет  **$O(1)$**  обновление счётчика и минимальную память.
2. Sliding log может иметь заметный рост состояния при высоком RPS и длинном окне.
3. Token Bucket имеет  **$O(1)$**  по вычислению, но требует точной синхронизации времени и состояния.
4. Leaky Bucket может требовать отдельной очереди, что влияет на задержку и мониторинг.

Поэтому сопоставление алгоритмов без учёта стоимости ресурсов методически некорректно: алгоритм может обеспечивать высокую точность, но быть избыточно дорогим в эксплуатации.

В практике промышленной эксплуатации выбор часто формируется как комбинация алгоритмов, а не как выбор конкретного «победителя».

Базовый алгоритм обслуживает штатный профиль, а защитный включается при обнаружении аномалий.

Именно эта логика затем проверяется в главе с экспериментом на проектном стенде.

### **Вывод по разделу 2.1.**

Для исследуемой задачи нецелесообразно фиксировать единственный алгоритм на постоянной основе.

Предпочтительно использовать гибридную политику: Token Bucket как базовый режим обычной эксплуатации и Sliding Window как усиленный режим для аномалий.

Это снижает риск неправильного выбора одного фиксированного алгоритма.

Дополнительно такой подход эффективнее и проще масштабируется: команда может поэтапно развивать политику, не перепроектируя весь контур допуска запросов.

## **2.2. Архитектурные модели интеграции механизмов ограничения запросов в распределённые системы**

Архитектурно механизмы лимитирования могут быть размещены в трёх точках: внутри сервиса, на входном шлюзе (API Gateway) и в выделенном централизованном сервисе.

На практике выбор точки интеграции определяет не только технические параметры, но и операционную модель команды: кто управляет политикой, как быстро вносить изменения, где наблюдать за метриками и как выполнять безопасный откат.

Поэтому архитектура лимитирования должна рассматриваться как часть платформенной архитектуры.

### **Встраивание в приложение.**

Преимущества: минимальная добавленная задержка, близость к бизнес-контексту, простая отладка.

Недостатки: труднее поддерживать единые лимиты при горизонтальном масштабировании без общего состояния.

Практические реализации часто используют Bucket4j и Resilience4j [22] [23].

Этот вариант хорош, когда требуется тонкая бизнес-логика ограничений:

например, разные лимиты для разных типов операций внутри одной конечной точки или учёт внутренних атрибутов запроса.

Но чем больше сервисов в системе, тем выше риск расхождения политик между командами.

Поэтому для мультисервисных платформ такой подход редко является единственным.

### **Лимитирование на уровне API-шлюза.**

Преимущества: единая точка конфигурации, унифицированные ответы 429, проще управлять внешними клиентами.

Недостаток такого варианта в том, что шлюз видит не весь прикладной контекст. Кроме того, некоторые лимиты срабатывают с небольшими отклонениями, потому что решение принимается в распределённой среде [3] [4] [14] [20].

Это оптимальный слой для первичного входного контроля.

Именно слой API-шлюза обычно первым отсекает массовую аномалию.

Он позволяет быстро изменить глобальный профиль без выпуска новых версий прикладных сервисов.

Однако из-за ограниченного бизнес-контекста API-шлюз плохо подходит для сложных доменных правил, поэтому требуется дополнительный прикладной слой.

Пример архитектуры API-шлюза с выделенным контуром лимитирования и хранилищем состояния показан на рисунке 3.

#### Архитектура API-шлюза с контуром ограничения запросов

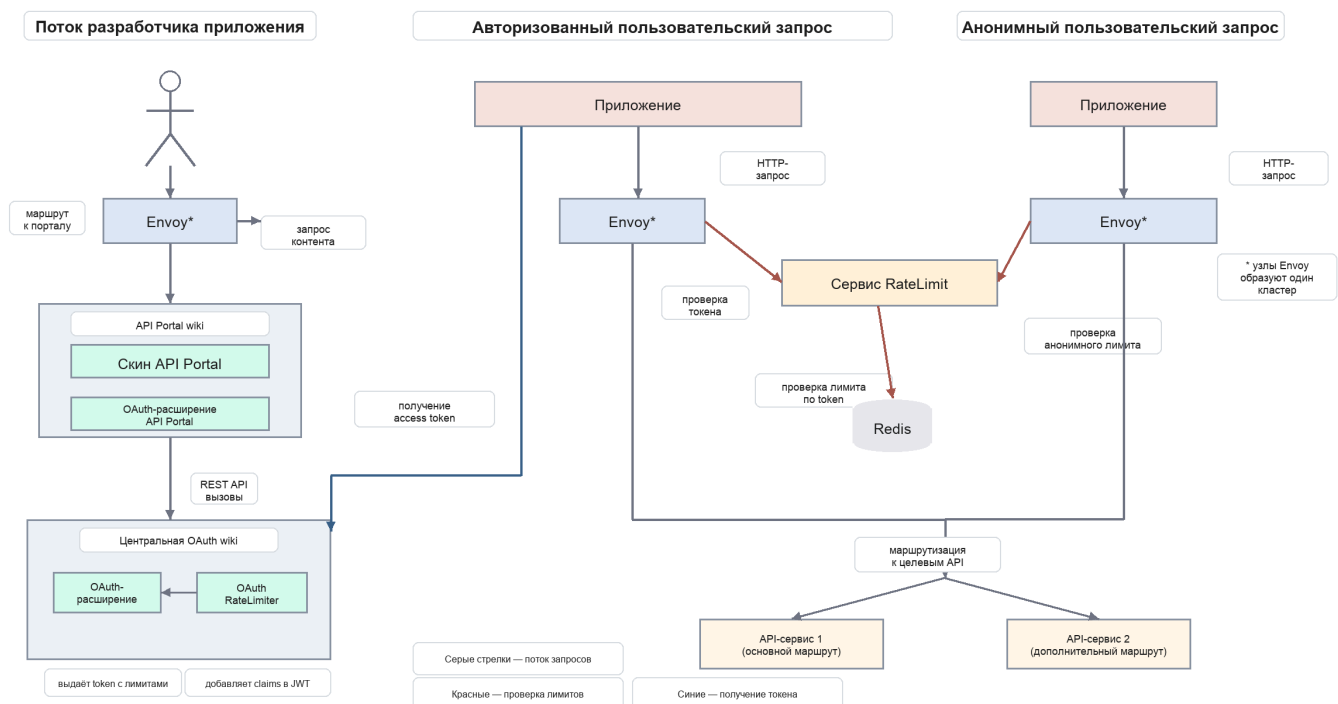


Рисунок 3 – Архитектура API-шлюза с контуром ограничения запросов

#### Централизованный сервис лимитирования.

Преимущества: единая логика, единая телеметрия, сквозной контроль в мультисервисной системе.

Недостатки: дополнительный сетевой переход, риск стать узким местом, необходимость реализации резервного режима.

Такой подход оправдан там, где консистентность политики важнее минимизации задержки каждого запроса.

В рассматриваемом проекте именно этот подход выбран как базовый для эксперимента, так как он позволяет сравнивать алгоритмы в сопоставимых условиях.

Дополнительный сетевой переход может быть оправдан, если важнее единая телеметрия и управляемая смена алгоритмов.

Для исследовательской задачи это критично: иначе влияние алгоритма и влияние места интеграции смешиваются, и интерпретация результатов становится неоднозначной.

### **Роль Redis и распределённого состояния.**

Redis часто используется как быстрое хранилище счётчиков и токенов.

Команда INCR и связанные паттерны подходят для счётчиков и оконных схем [24].

Однако в Redis Cluster используется асинхронная репликация и возможны окна потери подтверждённых записей при разделении сети [5].

Эта особенность должна учитываться в проектировании требований к поведению лимитера при сбоях.

Практическая архитектурная мера здесь состоит в явной фиксации допустимой погрешности.

Если нужно гарантировать полное отсутствие превышения лимита, приходится усложнять синхронизацию между узлами. Это повышает задержку и стоимость работы системы.

При допущении погрешности на коротких интервалах можно добиться сохранения доступности и приемлемой задержки.

Именно этот компромисс в дальнейшем проверяется на экспериментальных сценариях нагрузки.

### **Наблюдаемость как часть архитектуры.**

При лимитировании без мониторинга становится трудно объяснить, почему запрос был отклонён.

Prometheus ориентирован на сбор метрик временных рядов и хорошо подходит для телеметрии отклонений, задержек и пропускной способности [25].

Для эксплуатации важно отслеживать не только число HTTP ответов 429, но и контекст: по ключам, маршрутам, периодам, типам клиентов.

Минимальный профиль для эксплуатации метрик включает:

1. **forwarded/total** и **reject\_percent** по алгоритму и сценарию.
2. **avg/p95/p99-задержка** на прокси и целевом сервисе.
3. Ошибки нагрузочного генератора и целевого сервиса.
4. Состояние распределённого хранилища лимитов.

Без этой информации невозможно обосновать, что конкретная настройка действительно повышает устойчивость, а не переносит проблему в другой участок системы.

С точки зрения отказоустойчивости для любой архитектуры существуют свои риски.

1. Лимитер внутри сервиса рискует тем, конфигурация может начать отличаться между командами.
2. Лимитер на шлюзе рискует потерей доменного контекста.
3. Централизованный лимитер рискует стать узким местом при резком росте нагрузки.

Итоговое решение должно включать не только сам алгоритм, но и стратегию поведения при отказе, то есть пропускать часть трафика или блокировать запросы, а также план деградации на случай недоступности Redis или управляющего контура.

В прототипе выбран режим, при котором основной сервис продолжает обрабатывать запросы, даже если часть вспомогательных компонентов временно недоступна.

Это позволяет сервису продолжать отвечать пользователям даже при частичной деградации управляющей инфраструктуры.

При приоритете непрерывности пользовательского доступа такой подход предпочтительнее стратегии полной блокировки при любой внутренней ошибке.

Для выбора архитектуры полезно сопоставлять варианты по единой матрице критериев; такое сопоставление представлено в таблице 4.

Таблица 4 – Сравнение архитектурных точек интеграции лимитирования.

| <b>Критерий</b>                      | <b>В приложении</b>               | <b>На API-шлюзе</b>          | <b>Централизованный сервис</b>               |
|--------------------------------------|-----------------------------------|------------------------------|----------------------------------------------|
| Задержка                             | Минимальная                       | Низкая                       | Выше из-за дополнительного сетевого перехода |
| Единообразие политики                | Низкое при большом числе сервисов | Высокое для внешнего трафика | Высокое для всей платформы                   |
| Доменный контекст                    | Максимальный                      | Ограниченный                 | Средний (зависит от передаваемых атрибутов)  |
| Скорость централизованного изменения | Низкая                            | Высокая                      | Высокая                                      |
| Риск единой точки отказа             | Низкий                            | Средний                      | Высокий                                      |

Из матрицы следует, что выбор зависит от задержки, контекста и риска единой точки отказа.

Если ключевой приоритет — единая управляемость и аналитика, рационален централизованный контур с резервированием.

Если ключевой приоритет — минимальная задержка и локальная бизнес-логика, рационально встраивание в сервисы.

В мультисервисных публичных платформах чаще всего оказывается эффективной комбинированная модель с распределением ответственности между слоями.

### **Вывод по разделу 2.2.**

В реальных системах наилучший эффект даёт комбинация слоёв:

1. Грубый входной контроль на API шлюзе.
2. Точечные прикладные ограничения в сервисах.
3. Централизованная аналитика и коррекция параметров.

Такой подход устойчивее к единичным ошибкам настройки и проще сопровождать при росте числа сервисов и команд.

Для текущей работы это также обеспечивает проверяемость: можно воспроизводимо менять параметры и фиксировать эффект в метриках одного стенда.



### **2.3. Компромиссы между точностью, отказоустойчивостью и адаптивностью механизмов ограничения запросов**

В распределённом лимитере несколько узлов одновременно принимают решения по одному общему лимиту. Чтобы считать запросы абсолютно точно, им нужно часто синхронизировать состояние счётчиков. Это увеличивает задержку и создаёт дополнительную нагрузку на хранилище или координатор. Если же узлы принимают решения самостоятельно, система работает быстрее и лучше переживает временные проблемы соединения, но в отдельные моменты возможны небольшие превышения лимита из-за рассогласования счётчиков. Поэтому на практике лимитер настраивают как компромисс между точностью ограничения, задержкой ответа и устойчивостью работы.

Этот компромисс является известным свойством распределённых систем.

Поэтому инженерный вопрос формулируется так: где именно провести границу допустимой ошибки, чтобы обычные пользователи реже получали отказ.

Если граница выбрана неверно, система либо слишком сильно защищается (много ложных HTTP ошибок 429), либо защищается плохо (рост HTTP ошибок 5xx и деградация нижестоящего компонента).

AWS и Azure прямо указывают, что лимиты следует трактовать как целевые значения, а не как математически абсолютный потолок в каждый момент [3] [4].

Это не недостаток документации, а отражение устройства распределённых систем.

Инженерно корректная стратегия — проектировать политику так, чтобы ошибки были ограничены по масштабу и времени. Другими словами ошибка должна быть дешёвой, в том числе для дальнейшего рефакторинга.

Это означает, что кратковременный пропуск части сверхлимитного трафика не должен приводить к системной аварии, а кратковременное ужесточение не должно массово блокировать легитимных пользователей.

Именно поэтому на практике обычно используют не один общий лимит, а несколько уровней ограничений. Например, отдельно задают общий лимит для всего API, лимит для конкретного клиента, лимит для отдельного маршрута и более строгие правила для критичных операций. Такой подход позволяет не блокировать весь трафик одинаково, а точнее управлять нагрузкой в зависимости от источника запроса и важности операции.

Redis Cluster в своей спецификации формулирует этот компромисс явно: асинхронная репликация, окна возможной потери записей, приоритет доступности при отказах [5].

Для механизма ограничения запросов в многозонной среде важно заранее определить допустимую погрешность и поведение системы при потере связи между узлами.

Для реального продукта полезно задать как минимум три профиля эксплуатации:

1. Штатный режим: акцент на доступности и минимизации ложных блокировок.
2. Предупредительный режим: умеренное ужесточение при признаках перегрузки.
3. Аварийный режим: жёсткая фильтрация для стабилизации системы.

Переходы между режимами должны выполняться по метрикам и порогам.

Ключевой вопрос формулируется не как достижение идеальной точности, а как определение допустимой ошибки без потери ключевых бизнес-процессов.

Для пользовательских API обычно предпочтителен режим, где часть сверхлимитных запросов может временно пройти, но сервис в целом остаётся доступным.

Для высококритичных операций (например, авторизация, платёжные шаги) допустим более строгий профиль.

Эта градация позволяет согласовать технические решения с бизнес-приоритетами.

Например, для справочных API можно использовать более мягкие лимиты, чтобы пользователи реже сталкивались с отказами. А для дорогих транзакционных операций лимиты должны быть строже, потому что каждый лишний запрос заметно нагружает систему или увеличивает стоимость обработки.

В результате система управляет риском дифференцированно, а не единым режимом для всех типов задач.

#### **Адаптивность как способ улучшить компромисс.**

Если лимиты изменяются по метрикам, система может заранее усиливать защиту перед ожидаемым пиком и ослаблять её в спокойные периоды.

Это уменьшает и ложные блокировки легитимного трафика, и риск плохой фильтрации во время аномального всплеска.

Внешний аналитический компонент здесь играет роль регулятора параметров, а не заменяет базовые алгоритмы.

Чтобы адаптивность не создавала дополнительной нестабильности, необходимы защитные ограничения:

1. Ограничение скорости изменения параметров.
2. Минимальное время удержания режима для исключения колебаний.
3. Границы параметров для безопасной эксплуатации.
4. Автоматический откат к базовому профилю при аномальном росте ошибок.

При наличии этих ограничений изменение лимитов не вызывает постоянных переключений режимов.

Отдельное архитектурное решение связано с политикой отказа:

1. Пропуск при сбое означает, что при отказе лимитера часть трафика проходит дальше. Приоритет у доступности.

2. Блокирование при сбое означает, что при отказе лимитера трафик блокируется. Приоритет у защиты.

Для публичных пользовательских API в большинстве случаев будет лучше контролируемый пропуск при сбое с дополнительными защитными ограничениями, поскольку полное блокирование при сбое может превратить внутреннюю проблему лимитера в массовую недоступность внешнего сервиса.

Для критичных операций может быть оправдана частичная блокировка при сбое, но только для ограниченного набора конечных точек (endpoints).

Корректный компромисс определяется через бизнес-классификацию маршрутов и целевые SLO (ожидаемый уровень качества ответа сервера).

Один и тот же алгоритм при разных целях может быть как удачным, так и неудачным.

Для закрепления этого тезиса полезно использовать матрицу критичности конечной точки и режима деградации, представленную в таблице 5.

Таблица 5 – Политика реакции по критичности операций.

| Класс операции           | Пример                         | Приоритет                   | Предпочтительный режим при риске перегрузки                |
|--------------------------|--------------------------------|-----------------------------|------------------------------------------------------------|
| Критичный транзакционный | Платёж, подтверждение личности | Целостность и защита        | Более строгий профиль и ограниченное блокирование при сбое |
| Важный пользовательский  | Авторизация, оформление заказа | Баланс доступности и защиты | Контролируемый пропуск при сбое и усиленные лимиты         |

|            |                                      |                             |                                                                |
|------------|--------------------------------------|-----------------------------|----------------------------------------------------------------|
| Справочный | Каталог, статус,<br>публичные данные | Максимальная<br>доступность | Мягкий профиль с<br>приоритетом на<br>пользовательский<br>опыт |
|------------|--------------------------------------|-----------------------------|----------------------------------------------------------------|

Такая матрица позволяет избежать ошибки «одинаковая политика на все маршруты».

При дифференцированном управлении система одновременно сохраняет доступность часто используемых функций и защищает критичные операции от недопустимых рисков.

Это напрямую повышает ценность решения для конечных пользователей и снижает вероятность бизнес-инцидентов из-за неверно выбранного универсального режима.

Статическая конфигурация хорошо работает только для заранее известного профиля нагрузки, который в реальности почти не существует.

Адаптивный контур потенциально позволяет улучшить баланс между точностью, устойчивостью и пользовательским опытом без радикального усложнения алгоритмического ядра.

Для исследуемого прототипа это особенно важно, поскольку трафик в сценариях эксперимента намеренно неоднороден; значит, преимущество должно оцениваться не в одной точке, а в динамике режимов.

Современные научные публикации 2021-2025 годов подтверждают актуальность такой проблемы. В российских рецензируемых работах, рассматриваются оптимизация программных систем и конфиденциальная обработка ИИ в облаках [26] [27], распределённое хранение [28], модели трафика, очередей и нагрузки в телекоммуникационных системах [29] [30] [33] [34], устойчивость передачи при атаках и защищённые коммуникации [31] [32], пропускная способность Wi-Fi и моделирование LoRaWAN [35] [36]. Эти работы важны для обоснования требований к наблюдаемости, анализу угроз и адаптивному реагированию.

Международные публикации, индексируемые в Scopus и/или Web of Science, показывают близкую исследовательскую линию: адаптивную

настройку правил обработки трафика, влияние распределённого ограничения запросов на сервисы, чувствительные к задержке, DDoS/DoS-защиту в SDN и облачных или контейнерных средах, а также применение ML/DL для обнаружения и смягчения атак [37] [38] [39] [40] [41] [42] [43] [44]. Поэтому экспериментальный стенд в этой работе проверяет не только алгоритмическую корректность лимитера, но и компромисс между доступностью, задержкой и защитной функцией.

### **Вывод по разделу 2.3.**

1. Универсально лучшего алгоритма не существует: выбор зависит от профиля нагрузки и архитектурных ограничений.
2. Наиболее практичным базовым вариантом для API-нагрузки выступает Token Bucket, особенно в комбинации с политиками API-шлюза [3] [20].
3. Для аномальных сценариев полезно иметь режим усиления (Sliding Window/ужесточение порогов), управляемый телеметрией.
4. Архитектурно оправдан гибрид: API-шлюз + сервисные лимиты + централизованный аналитический слой.

## Глава 3. Экспериментальное исследование алгоритмов ограничения запросов в распределённой системе

### 3.1. Методика проведения эксперимента и описание архитектуры прототипа системы

Для эксперимента был реализован прототип. Его архитектура включает:

1. Сервис генерации нагрузки.
2. Прокси/лимитер с поддержкой нескольких алгоритмов.
3. Целевой сервис обработки запросов.
4. Redis для разделяемого состояния лимитера.
5. Контур мониторинга и сбор метрик.

Логика маршрутизации организована так, что каждый входящий запрос проходит через слой лимитирования до попадания в целевой сервис.

Это позволяет объективно измерять влияние выбранного алгоритма на допуск трафика, задержку и устойчивость целевого узла.

Мониторинг построен на сборе метрик временных рядов: задержка прокси, доля отклонений, число успешных проходов, эффективный RPS [25].

Для аналитики использованы локальные файлы проекта с результатами прогонов: матрица сценариев и расчётные оценки [45] [46] [47].

**Ключевой принцип методики — воспроизводимость.**

Все прогоны выполняются одинаковыми скриптами, с одинаковыми базовыми параметрами лимитов и фиксированными сценариями генерации трафика.

Это позволяет сравнивать алгоритмы по разнице поведения в одной и той же среде.

Структура представления и обработки экспериментальных результатов по сценариям показана на рисунке 4.

### Структура обработки экспериментальных результатов



Рисунок 4 – Структура экспериментальных результатов по сценариям

Базовый сценарий сравнения (статический режим) запускается скриптом **scripts/battle\_matrix.sh** (репозиторий: [https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/battle\\_matrix.sh](https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/battle_matrix.sh)) с параметрами:

1. Длительность одного прогона — 12 секунд.
2. На каждую комбинацию «сценарий + алгоритм» выполняется 10 повторов.
3. Рандомизация порядка алгоритмов внутри сценария для снижения эффекта прогрева.
4. Единый **base\_rps\_limit=100** и **window=10s** для сопоставимых стартовых настроек [46].

Дополнительная адаптивная серия выполняется скриптом **scripts/adaptive\_phase\_benchmark.sh** (репозиторий: [https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/adaptive\\_phase\\_benchmark.sh](https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/adaptive_phase_benchmark.sh)).

Она построена как фазовый эксперимент, а не как короткое попарное сравнение:

1. сценарий всплеска с восстановлением: **30s constant(40 RPS) -> 30s burst\_stress -> 30s constant(40 RPS)**.



2. сценарий DDoS с восстановлением: 30s constant(40 RPS) -> 30s ddos -> 30s constant(40 RPS).

3. 6 повторов, параллельность 256, начальный режим адаптивного контура — Token Bucket.

4. Тем самым в эксперименте разделены две задачи: статическая матрица сравнивает fixed, token и sliding при неизменной политике, а фазовая адаптивная серия проверяет, как отдельный модуль управления переключает режим между Token Bucket и Sliding Window [48] [49] [50] [51] [52].

Адаптивный модуль реализован как отдельный сервис-советник.

Он не принимает финальное решение вместо лимитера, а рекомендует параметры (например, изменение ёмкости/скорости пополнения или порога окна) на основании наблюдаемой динамики.

В прототипе для прогнозного слоя используется Prophet как готовый инструмент для временных рядов [6].

С точки зрения механики прогноза Prophet работает с временным рядом наблюдаемой нагрузки, где каждой точке ставится в соответствие момент времени и измеренное значение интенсивности потока запросов.

В реализации прототипа такими точками выступают пары «timestamp — observed RPS», которые передаются во внешний аналитический модуль и преобразуются в стандартный для Prophet формат **ds** и **y**.

В общем случае Prophet раскладывает данные на несколько частей: общий рост или спад, повторяющиеся закономерности и случайные отклонения. В прототипе суточная, недельная и годовая повторяемость отключены, так как стендовые эксперименты короткие и для них важнее ближайшее изменение нагрузки, а не долгие календарные циклы.

После подгонки модели по накопленной истории модуль формирует прогноз на один горизонт вперёд; в текущей конфигурации это 60 секунд.

Из результата используется значение **yhat** как ожидаемый RPS на ближайшем интервале, после чего оно сопоставляется с текущими метриками

**rejectedRate**, **latencyP95** и **errors5xx** и участвует в выборе режима **token**, **sliding** или сохранения текущей политики.

Если Prophet недоступен, история слишком короткая или построение прогноза завершилось ошибкой, применяется резервный сценарий: берётся усреднение по последним наблюдениям.

В работе Prophet используется как готовый инструмент краткосрочного прогноза временного ряда, а не как автономный механизм, который самостоятельно принимает решение.

Работа не включает разработку новой ML-модели и соревнование алгоритмов машинного обучения.

Здесь проверяется эффект внедрения прогнозного сигнала в уже существующий контур управления лимитированием.

Это разграничение важно для корректной интерпретации результатов.

Эксперимент отвечает на вопрос: меняется ли поведение сервиса при подключении адаптивного слоя в воспроизводимых сценариях нагрузки.

Оценка качества отдельных ML-моделей в предмет работы не входит.

Сервисная топология стенда воспроизводит упрощённую схему продуктового API сервиса:

1. Входной поток формируется генератором нагрузки.
2. Прокси-лимитер применяет политику допуска и возвращает HTTP код 429 при необходимости.
3. Разрешённые запросы передаются в целевой сервис.
4. Состояние лимитов хранится в Redis.
5. Метрики собираются через **actuator/prometheus**.

Такое разделение критично для корректности эксперимента.

Если генерация, лимитирование и целевая обработка смешаны в одном процессе, сложно отделить работу алгоритма от эффекта конкурентного исполнения внутри приложения.

В текущем стенде эта проблема уменьшена архитектурно за счёт разделения ролей сервисов.

Сценарии тестирования:

1. Низкая постоянная нагрузка.
2. Синусоидальная нагрузка.
3. Пуассоновский поток.
4. Высокая постоянная нагрузка.
5. Короткие всплески.
6. DDoS-нагрузка.

Параметры сценариев зафиксированы в скриптах бенчмарка и используются в одинаковом виде для всех алгоритмов; их сводка приведена в таблице 6.

Таблица 6 – Профили сценариев нагрузки (из конфигурации скриптов).

| Сценарий           | Параметры профиля                                          | Назначение                                                      |
|--------------------|------------------------------------------------------------|-----------------------------------------------------------------|
| низкая постоянная  | rps=40                                                     | Проверка отсутствия лишних отклонений в спокойном режиме        |
| синусоидальная     | minRps=15, maxRps=170, period=12s                          | Проверка реакции на плавные циклические колебания               |
| пуассоновская      | averageRps=140                                             | Проверка стохастического потока без регулярной формы            |
| высокая постоянная | rps=180                                                    | Проверка поведения при устойчивой нагрузке выше базового лимита |
| всплеск            | baseRps=20, spikeRps=240, spikeDuration=2s, spikePeriod=8s | Проверка обработки коротких всплесков                           |
| DDoS               | minRps=35, maxRps=320, maxSpikeDuration=2s                 | Проверка защитной функции в аномальном режиме                   |

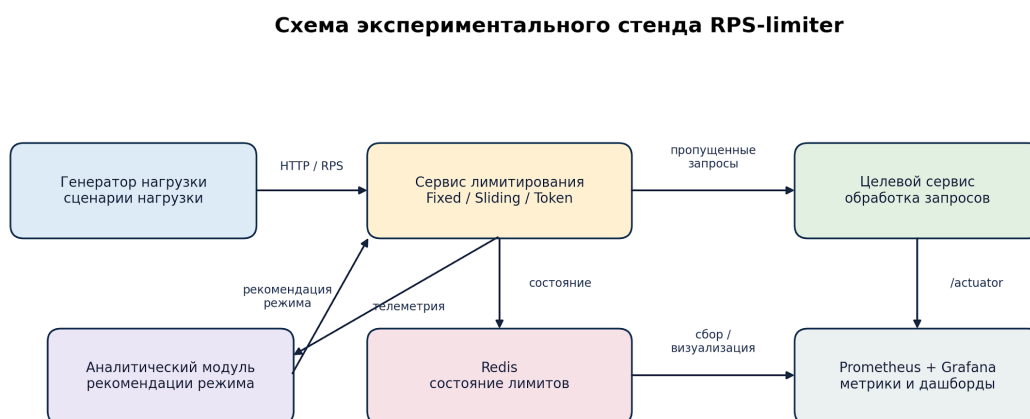
Подбор этих профилей позволяет покрыть разные практические режимы эксплуатации.

**constant\_low** и **sinusoidal** моделируют штатную пользовательскую активность.

**poisson** отражает нерегулярный фоновый поток.

**burst** и **ddos** имитируют сценарии, где особенно важна устойчивость к резким скачкам входящего трафика.

Структура экспериментального стенда, используемого в разделе 3.1, представлена на рисунке 5.



Стенд воспроизводит сервисы проекта, управляющий контур и измерение реальных метрик.

Рисунок 5 – Схема экспериментального стенда

Для обеспечения сопоставимости алгоритмов применяется единая схема конфигурирования:

1. Для Fixed Window и Sliding Window лимит задаётся как произведение базового RPS-лимита и размера окна.
2. Для Token Bucket задаются скорость пополнения и ёмкость бакета, равная двум базовым RPS-лимитам.
3. Перед каждым запуском сценария выполняется принудительное обновление конфигурации лимитера.

Это устраняет распространённую ситуацию, когда один алгоритм получает «более мягкие» стартовые параметры и выигрывает не из-за свойств модели, а из-за конфигурации.

В работе сравнение строится именно как сравнение поведения при согласованном базовом бюджете.

Измеряемые метрики берутся из CSV-результатов скриптов и включают:

1. **total\_requests, forwarded, rejected.**
2. **reject\_percent, effective\_rps.**
3. **Средняя, p95- и p99-задержка прокси в миллисекундах.**
4. **loadgen\_errors, error\_percent.**
5. В адаптивной серии дополнительно: **число переключений**, длительность работы в Token Bucket и Sliding Window, фазовые значения **reject\_percent** и **effective\_rps**.

Выбор метрик связан с задачей сравнить скорость (latency) и надёжность (доступность для штатного трафика + фильтрация в DDoS) отдельно друг от друга.

Если объединять всё в один агрегированный балл без контекста, можно получить методологическую ошибку интерпретации:

алгоритм с жёсткой блокировкой атаки может демонстрировать формально более высокую итоговую оценку даже при ухудшении пользовательской доступности в штатных режимах.

Для повышения статистической устойчивости интерпретации используются повторные прогоны и интервальные оценки.

В рабочих артефактах проекта сохранены агрегаты со средними значениями и CI95, что позволяет видеть не только «центральное значение», но и разброс результатов [47] [50].

Это особенно важно как для коротких статических окон (12 секунд), так и для фазовых адаптивных прогонов, где значение имеет не только общий итог, но и распределение эффекта между нормальной, атакующей и восстановительной фазами.

### **Критерии успешности эксперимента.**

Критерии успешности эксперимента фиксируются заранее, чтобы выводы опирались не на интуитивную оценку графиков, а на измеримые условия приёма. В этой работе успешность не сводится к одному лучшему показателю: необходимо одновременно учитывать сохранение пользовательской доступности, усиление защитной функции в аномальной нагрузке и приемлемую стоимость этого решения по задержке.

1. Для статической матрицы успешным считается результат, при котором алгоритм в штатных сценариях сохраняет высокую долю **forwarded/total**, не вызывает значимого роста **error\_percent**, **avg/p95 latency** и не переводит прокси или генератор в нестабильный режим сопровождающийся сбоями.

2. Для аномальных сценариев успешность оценивается по способности лимитера увеличивать **reject\_percent**, снижать давление на целевой сервис и тем самым предотвращать каскадную деградацию, но без перехода самого стенда в режим недостоверных измерений из-за критичного роста внутренних ошибок.

3. При интерпретации требуется отдельная оценка доступности для легитимного трафика, защитной функции при аномальной нагрузке и стоимости этой защиты по задержке.

4. Для адаптивной серии успешность определяется дополнительно по фазовому поведению. В нормальной фазе режим должен сохранять доступность на уровне лучшего статического варианта, в атакующей фазе должен наблюдаться управляемый переход к более подходящему профилю Token Bucket или Sliding Window, а в таймлайне должны фиксироваться реальные переключения алгоритмов и параметров, а не только изменение чисел в конфигурации.

5. Для восстановительной фазы успешным считается снижение **reject** после окончания атаки и тенденция возврата к базовому профилю. Если возврат происходит частично и защита остаётся чрезмерно строгой, результат трактуется как ограниченно успешный: сам контур считается

работоспособным, но его гистерезис и пороги требуют дополнительной настройки.

6. Любой вывод признаётся обоснованным только при наличии повторов и интервальной оценки **CI95**; единичный прогон рассматривается как иллюстрация, но не как достаточное доказательство, поскольку кратковременные серии чувствительны к случайному разбросу и особенностям конкретного окна наблюдения.

Успешным результатом эксперимента считается получение воспроизводимых данных, которые позволяют понять, какой компромисс между доступностью, защитой и задержкой является приемлемым для разных профилей нагрузки.

Отдельно фиксируются угрозы валидности эксперимента:

1. Ограниченная длительность одного прогона может недооценивать эффекты медленной адаптации.
2. Локальный стенд не воспроизводит в полном объёме сетевую неоднородность многорегионального продакшна.
3. Отдельный анализ чувствительности к вариациям аппаратных ресурсов не выполнялся.

Тем не менее, для целей диссертации выбранная методика достаточна, так как она:

1. Основана на реальном исполняемом коде текущего проекта.
2. Воспроизводима по командам и артефактам.
3. Позволяет сопоставить алгоритмы на едином наборе сценариев.
4. Даёт метрики, пригодные для дальнейшего использования.

В работе зафиксирован регламент воспроизведения эксперимента, приведённый в таблице 7.

Таблица 7 – Регламент выполнения серии прогонов.

| Шаг | Действие | Проверяемый результат |
|-----|----------|-----------------------|
|-----|----------|-----------------------|

|   |                                                                      |                                                                                                    |
|---|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1 | Проверка доступности сервисов стенда                                 | <b>actuator/health</b> всех компонентов в состоянии UP                                             |
| 2 | Запуск матрицы статических прогонов ( <b>battle_matrix.sh</b> )      | Сформированы <b>raw.csv</b> , <b>summary.csv</b> , <b>overall.csv</b>                              |
| 3 | Проверка качества данных (нулевые/аномальные строки)                 | Валидная выборка без критичных ошибок генератора                                                   |
| 4 | Запуск фазовой adaptive-серии ( <b>adaptive_phase_benchmark.sh</b> ) | Сформированы <b>raw.csv</b> , <b>summary.csv</b> , <b>switch-summary.csv</b> , <b>timeline.csv</b> |
| 5 | Построение графиков из CSV                                           | Сгенерированы PNG с фазовой долей отклонений и таймлайном переключений                             |
| 6 | Интерпретация с учётом CI95                                          | Выводы учитывают разброс, а не только средние значения                                             |

Регламент обеспечивает повторяемость результатов при повторной проверке и задаёт основу для автоматизации регулярных прогонов в конвейере непрерывной интеграции.

Дополнительно для корректности интерпретации учитываются методические правила:

1. Не сравнивать алгоритмы по одному агрегированному показателю без разбиения на показатели «скорости» и «надёжности».
2. Не делать вывод по единственному прогону, особенно на коротком временном окне.
3. Проверять, не обусловлен ли результат артефактом конфигурации, а не поведением алгоритма.
4. Фиксировать ограничения валидности.



### 3.2. Реализация и сравнение алгоритмов ограничения запросов в условиях различных сценариев нагрузки

Основной набор прогонов для матрицы выполнен в статическом режиме (**adaptive=off**) скриптом **scripts/battle\_matrix.sh** (репозиторий: [https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/battle\\_matrix.sh](https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/battle_matrix.sh)) с **duration=12s**, **repeats=10** и рандомизацией порядка алгоритмов на сценариях **constant\_low**, **sinusoidal**, **poisson**, **constant\_high**, **burst**, **ddos** [46].

Чтобы выводы отражали насколько отличается поведение алгоритмов, оценка разделена на два независимых критерия:

1. Скорость: средняя задержка прокси, где меньшее значение соответствует лучшему результату.

2. Надёжность: отдельно для штатной нагрузки и для DDoS.

Для штатной нагрузки (низкая постоянная, синусоидальная, пуассоновская, высокая постоянная и всплеск) использована доля успешно пропущенных запросов **forwarded/total**.

Для DDoS использована доля отфильтрованных запросов при контроле ошибок (**loadgen\_errors**).

Показатель **score** оставлен только как дополнительная справка. Его нельзя использовать как главный критерий, потому что высокая степень отклонений в DDoS может повысить итоговую оценку и одновременно ухудшить доступность обычного трафика.

Скоростные результаты сведены в таблице 8.

Таблица 8 – Скорость (статический режим, длительность 12 с, 10 повторов, среднее  $\pm$  CI95).

| Алгоритм       | Средняя задержка, мс | Задержка в DDoS, мс |
|----------------|----------------------|---------------------|
| Sliding Window | 1.436 $\pm$ 0.097    | 0.902 $\pm$ 0.062   |
| Fixed Window   | 1.523 $\pm$ 0.112    | 1.035 $\pm$ 0.077   |
| Token Bucket   | 1.606 $\pm$ 0.126    | 1.172 $\pm$ 0.142   |

Сопоставление скоростных характеристик алгоритмов визуализировано

на

рисунке

6.

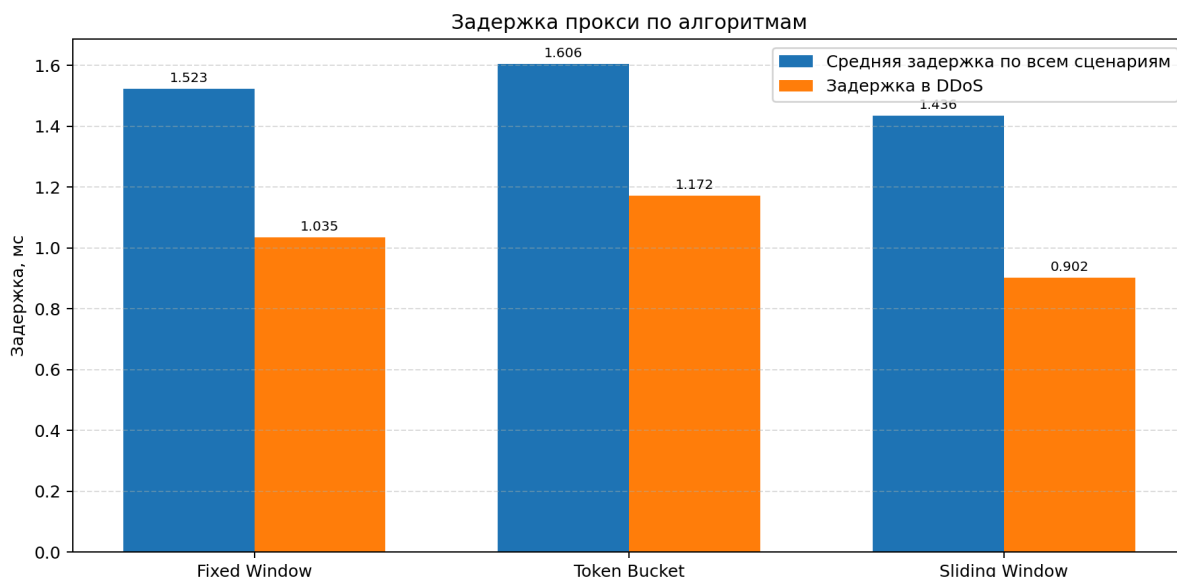


Рисунок 6 – Задержка прокси по алгоритмам

Показатели надёжности в статическом режиме приведены в таблице 9.

Таблица 9 – Надёжность (статический режим, длительность 12 с, 10 повторов, среднее  $\pm$  CI95, 01.03.2026).

| Алгоритм       | Успех штатного трафика, % | Отклонения штатного трафика, % | Отклонения DDoS, % | Ошибки генератора, % |
|----------------|---------------------------|--------------------------------|--------------------|----------------------|
| Token Bucket   | 94.87 $\pm$ 1.76          | 3.21 $\pm$ 1.76                | 30.22 $\pm$ 5.22   | 0.00                 |
| Fixed Window   | 94.57 $\pm$ 1.89          | 3.52 $\pm$ 1.89                | 22.42 $\pm$ 6.75   | 0.00                 |
| Sliding Window | 88.01 $\pm$ 3.50          | 7.92 $\pm$ 3.50                | 42.59 $\pm$ 6.72   | 0.00                 |

Сравнение доступности в штатных режимах и доли DDoS-фильтрации представлено на рисунке 7.

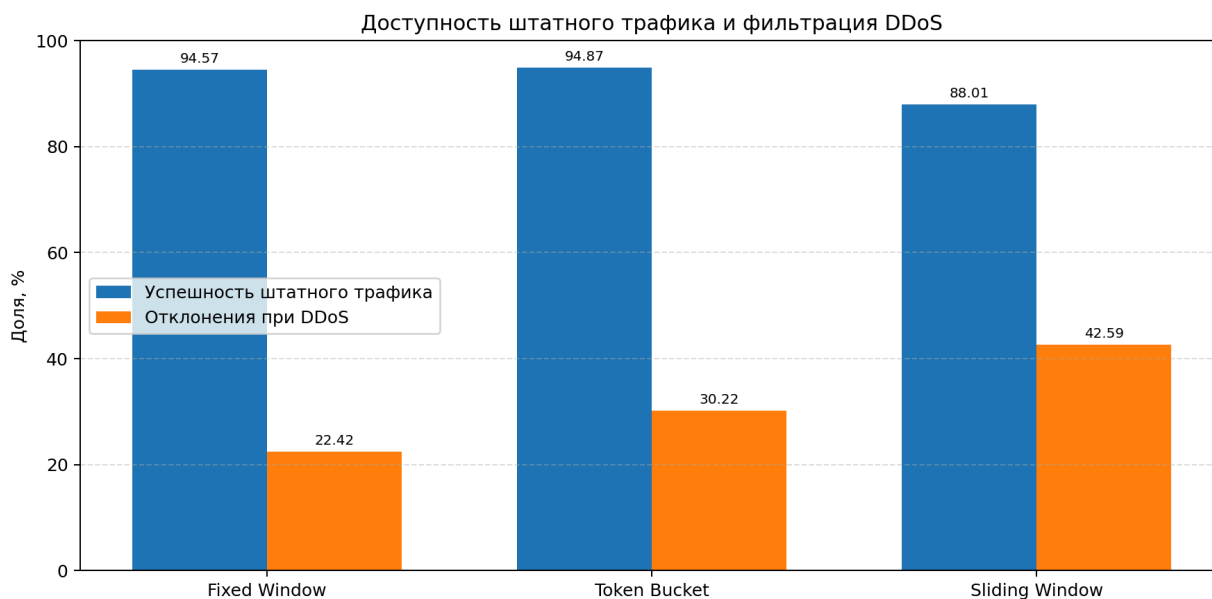


Рисунок 7 – Надёжность: успешность штатного трафика и отклонения при DDoS

По результатам серии прогонов получено следующее:

1. Самый быстрый алгоритм в текущей реализации — **sliding** (минимальная задержка).
2. По доступности штатного трафика **token** и **fixed** близки (94.87% и 94.57% соответственно), при этом интервалы CI95 пересекаются.
3. Самая жёсткая фильтрация в DDoS — у **sliding** (42.59%), но это достигается ценой более агрессивных отклонений в штатном режиме.
4. Нельзя назвать один алгоритм самым надёжным во всех сценариях: надёжность зависит от типа нагрузки и цели (доступность легитимного трафика или максимальная фильтрация атаки).

Что касается результатов Fixed Window в сценариях низкой постоянной нагрузки, синусоидальной нагрузки и всплеска алгоритм не создаёт лишних отклонений, а при высокой постоянной нагрузке сохраняет 86.29% успешных запросов, что немного выше Token Bucket.

Негативная сторона состоит в грубости окна: при лимите 1000 запросов на 10 секунд контролируется суммарный объём внутри окна, но слабее ограничивается мгновенная концентрация запросов на его границе.

Поэтому Fixed Window в такой постановке выглядит «надёжным» для клиента, однако эта надёжность достигается ценой более мягкой защиты системы, что подтверждается наименьшей долей DDoS-фильтрации среди трёх алгоритмов — 22.42%.

Результаты Sliding Window соответствуют его теоретической роли наиболее строгого контроллера плотности трафика.

Алгоритм показывает минимальную среднюю задержку и максимальную долю отклонений в DDoS — 42.59%, поскольку сглаженная оценка по текущему и предыдущему окну быстрее реагирует на рост интенсивности запросов и уменьшает эффект границы окна.

Одновременно такая точность делает его наиболее жёстким к легитимным колебаниям нагрузки: в **poisson** средний reject достигает 12.17%, а при высокой постоянной нагрузке — 27.44%.

В текущем стенде Sliding Window лучше всего подходит для усиленного защитного режима, но не как универсальный базовый профиль публичного API.

Результаты Token Bucket в наибольшей степени соответствуют ожидаемому поведению с точки зрения его доступности и защиты.

По средним показателям штатных сценариев именно этот алгоритм обеспечивает наилучшую доступность (94.87%) и минимальный средний процент отклонений (3.21%), особенно в пуассоновском сценарии, где успешность достигает 99.27%.

Это объясняется самой природой Token Bucket: кратковременные всплески допускаются за счёт накопленных токенов, после чего поток выравнивается по скорости пополнения.

Вместе с тем в DDoS алгоритм уступает Sliding Window по жёсткости фильтрации и оказывается ближе к Fixed Window, поскольку в текущей реализации используется общий бакет для всего входящего потока, а скорость пополнения и ёмкость бакета подбирались с приоритетом сохранения высокой доступности для добросовестных пользователей.

Token Bucket в этой работе выступает наиболее сбалансированным компромиссом для переменного легитимного трафика.

Следует учитывать и ограничение эксперимента.

Целевой сервис моделирует предсказуемую обработку с искусственной задержкой, но без сложной деградации нижестоящего компонента при перегрузке.

По этой причине избыточный пропуск трафика в Fixed Window не превращается в резкий рост ошибок, а проявляется прежде всего как более мягкая защитная политика.

Распределение доли отклонённых запросов по сценариям и алгоритмам приведено на рисунке 8.

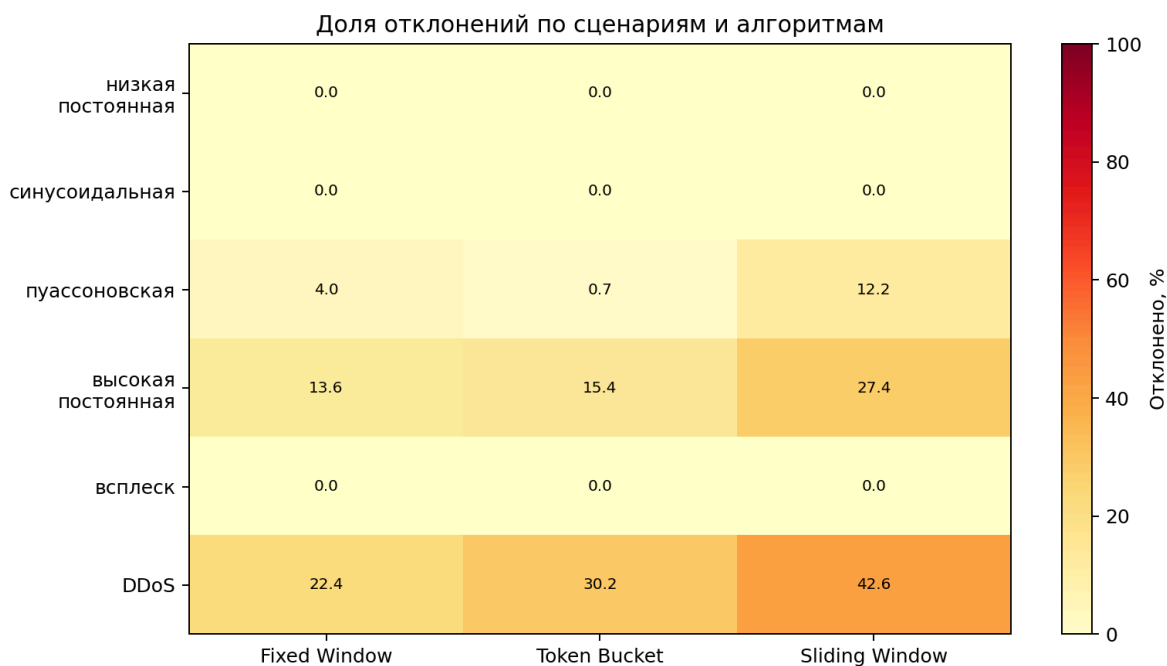


Рисунок 8 – Карта отклонений по сценариям и алгоритмам

По отдельным сценариям видна картина, которая позволяет точнее понять область применимости каждого алгоритма.

Сценарий низкой постоянной нагрузки показывает базовую корректность политики.

Все три алгоритма пропускают 100% запросов без отклонений, поскольку интенсивность нагрузки существенно ниже установленного бюджета.

Это означает, что в спокойном режиме различия между алгоритмами практически не влияют на доступность, а выбор должен делаться не по этому сценарию, а по поведению на более сложных профилях.

Синусоидальный сценарий также не создаёт отказов ни для одного алгоритма.

Плавные колебания в пределах заданного периода не истощают лимиты критическим образом, поэтому различия сводятся в основном к незначительным колебаниям задержки.

Практически это означает, что для предсказуемых циклических пиков любой из рассмотренных алгоритмов может использоваться как базовый уровень защиты.

Пуассоновский сценарий является наиболее показательным для оценки качества работы с естественно нерегулярным трафиком.

Именно здесь наиболее заметно проявляется преимущество Token Bucket: успешность достигает 99.27%, тогда как у Fixed Window он составляет 95.91%, а у Sliding Window снижается до 87.72%.

Объяснение состоит в том, что стохастический поток содержит кратковременные локальные всплески без строгой периодичности; Token Bucket сглаживает их за счёт накопленного запаса токенов, тогда как Sliding Window интерпретирует часть этих колебаний как повод для более раннего отклонения.

Для пользовательского API с нерегулярной, но штатной активностью Token Bucket в текущем стенде оказывается наиболее практичным выбором.

Сценарий высокой постоянной нагрузки моделирует устойчивую перегрузку выше базового бюджета и поэтому лучше всего показывает различие защитных стратегий.

Fixed Window сохраняет наибольшую доступность (86.29%), Token Bucket даёт близкий, но более строгий результат (84.52%), а у Sliding Window успешность снижается до 72.31%.

Здесь важно, что более высокая успешность у Fixed Window не означает, что он лучше управляет нагрузкой: алгоритм просто допускает большую плотность трафика на границе окна.

Token Bucket в этом режиме ведёт себя ожидаемо: после быстрого расходования стартового резерва он переходит к ограничению по скорости пополнения, поэтому защищает систему строже, чем Fixed Window, но мягче, чем Sliding Window.

Сценарий всплеска показал 100% успех для всех трёх алгоритмов, что на первый взгляд может выглядеть неожиданно.

В выбранной настройке теста всплески были слишком короткими и суммарно не перегрузили стенд на всём интервале наблюдения.

Поэтому различия между алгоритмами в этом сценарии почти не проявились, а сам результат следует интерпретировать не как отсутствие различий между подходами вообще, а как признак того, что конкретный профиль всплеска оказался для стенда недостаточно жёстким.

DDoS-сценарий наиболее явно разделяет алгоритмы по защитной функции алгоритмов.

Sliding Window отклоняет 42.59% запросов и тем самым обеспечивает максимальную фильтрацию аномального трафика; Token Bucket даёт промежуточный результат 30.22%; Fixed Window отклоняет только 22.42%.

Практический смысл этих чисел состоит в том, что Sliding Window лучше удерживает форму потока под атакой и сильнее разгружает целевой сервис, тогда как Fixed Window оставляет больше шансов на прохождение агрессивных серий запросов.

В аномальном режиме или режиме инцидента именно Sliding Window является наиболее оправданным защитным профилем, а Token Bucket — компромиссным вариантом, если приоритетом остаётся сохранение части легитимного трафика даже во время атаки.

Для оценки адаптивного контура выполнена отдельная серия фазовых экспериментов скриптом `scripts/adaptive_phase_benchmark.sh`

(репозиторий:

[https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/adaptive\\_phase\\_benchmark.sh](https://github.com/Islam-Y/RPS-limiter/tree/main/services/scripts/adaptive_phase_benchmark.sh)) [48] [49] [50] [51].

Сравнивались три режима: **static\_token**, **static\_sliding** и **adaptive**, где внешнему аналитическому модулю разрешено применять рекомендации адаптивного контура к сервису лимитирования.

Короткие прогоны  $\text{duration}=12\text{s}$  показывают только общий итог. Фазовый эксперимент дополнительно показывает, когда система переключает режим, насколько быстро выходит из защиты и не начинает ли часто менять алгоритмы без необходимости.

Таблица 10 – Фазовый сценарий «всплеск и восстановление» (20.03.2026, 6 повторов, среднее  $\pm$  CI95).

| Режим                | Успех в норме, %  | Откл. в атаке, % | Откл. при восстановлении, % | Переключение | Время Token Bucket / Sliding Window, с |
|----------------------|-------------------|------------------|-----------------------------|--------------|----------------------------------------|
| стат. Token Bucket   | 100.00 $\pm$ 0.00 | 24.31 $\pm$ 0.29 | 0.00 $\pm$ 0.00             | 0.00         | 90.0 / 0.0                             |
| адаптивный           | 100.00 $\pm$ 0.00 | 48.17 $\pm$ 3.42 | 8.56 $\pm$ 3.71             | 2.00         | 47.0 / 38.8                            |
| стат. Sliding Window | 100.00 $\pm$ 0.00 | 73.57 $\pm$ 5.94 | 0.51 $\pm$ 0.90             | 0.00         | 0.0 / 90.0                             |

В сценарии всплеска с восстановлением адаптивный режим в нормальной фазе полностью совпадает с Token Bucket: лишних отклонений нет, успешность сохраняется на уровне 100%.

В атакующей фазе **адаптивный режим повышает** долю отклонений до 48.17%, то есть заметно усиливает защиту по сравнению с **static\_token** (24.31%), но не доходит до жёсткости **static\_sliding** (73.57%).

Следовательно, модуль действительно меняет алгоритм при всплеске, а не оставляет лимитер в исходном режиме.



В то же время фаза восстановления выявляет ограничение текущей политики: **адаптивный режим удерживает 8.56%** отклонений даже после завершения этапа всплеска, тогда как **static\_token** сразу возвращается к нулевым отклонениям.

В сценарии всплеска внешний аналитический модуль уже доказал способность обнаруживать события и переключать режим, но механизм гистерезиса и защитные ограничения пока слишком консервативны для быстрого возврата к «мягкому» профилю после сравнительно короткой аномалии.

Таблица 11 – Фазовый сценарий «DDoS и восстановление» (20.03.2026, 6 повторов, среднее  $\pm$  CI95).

| Режим                   | Успех в<br>норме, % | Откл. в<br>атаке, % | Откл. при<br>восстановле<br>нии, % | Переключе<br>ния | Время<br>Token<br>Bucket /<br>Sliding<br>Window, с |
|-------------------------|---------------------|---------------------|------------------------------------|------------------|----------------------------------------------------|
| стат. Token<br>Bucket   | 100.00 $\pm$ 0.00   | 51.55 $\pm$ 2.25    | 0.00 $\pm$ 0.00                    | 0.00             | 90.0 / 0.0                                         |
| адаптивный              | 100.00 $\pm$ 0.00   | 71.04 $\pm$ 0.33    | 27.24 $\pm$ 3.78                   | 2.00             | 41.8 / 43.8                                        |
| стат. Sliding<br>Window | 100.00 $\pm$ 0.00   | 89.27 $\pm$ 1.15    | 21.35 $\pm$ 4.63                   | 0.00             | 0.0 / 90.0                                         |

В сценарии DDoS с восстановлением результат ещё нагляднее.

В нормальной фазе **adaptive** снова совпадает с Token Bucket, сохраняя полную доступность сервиса.

В атакующей фазе доля reject возрастает до 71.04%, что существенно выше **static\_token** (51.55%) и заметно ближе к защитному профилю **static\_sliding** (89.27%).

Следовательно, модуль полезен как координатор режима: при аномальной нагрузке он переводит систему к более строгой политике.

Однако и здесь восстановительная фаза остаётся слабым местом: **адаптивный режим возвращается к Token Bucket** только ближе к концу

сценария, поэтому в режиме восстановления сохраняется 27.24% отклонений, что хуже как **static\_token**, так и **static\_sliding**.

Практически это означает, что ранняя версия адаптивной политики хорошо решала задачу входа в защитный режим, но была недостаточно точной по выходу из него.

Этот результат стал основанием для дальнейшей инженерной доработки контура.

Дальнейшая оптимизация выполнялась не как обучение новой ML-модели, а как настройка адаптивного селектора и параметров исполнительного алгоритма.

В итоговой версии из adaptive-пула исключён **fixed**: экспериментально он оказался слишком мягким в DDoS и потому оставлен только для ручного сравнения.

Рабочий пул ограничен алгоритмами Sliding Window и Token Bucket, а внешний модуль управляет не только выбором алгоритма, но и параметрами Token Bucket (**fillRate**, **capacity**).

Ключевые изменения состояли в следующем:

1. Добавлен ранний переход из Sliding Window в Token Bucket при шумной перегрузке, чтобы контур не проигрывал статическому Token Bucket в сценариях, похожих на пуассоновский поток сценариях.

2. Добавлен кратковременное смягчение входа: в первые секунды после входа в Token Bucket параметры бакета временно настраиваются мягче, чтобы не терять легитимный стохастический трафик.

3. Для DDoS-сценариев добавлено защитное условие: система не возвращается из Token Bucket в Sliding Window сразу после снижения нагрузки, а ждёт устойчивого восстановления и выдерживает минимальное время в защитном режиме.

4. В Prometheus/Grafana добавлены метрики применяемого алгоритма, конфигураций и реальных переключений, чтобы отличать рекомендацию аналитического модуля от действительно применённого состояния лимитера.

Финальная проверка выполнена на смешанном фазовом сценарии 24.04.2026.

Сценарий включает последовательность из следующих шагов: стабильная нагрузка, пуассоновский поток, всплеск, DDoS, восстановление. Поэтому он проверяет универсальность адаптивного контура, а не победу в одном специально подобранном режиме.

Результаты приведены в таблице 12 [51].

Таблица 12 – Смешанный фазовый сценарий для финального адаптивного режима (24.04.2026, 3 повтора).

| Фаза        | Успех адапт., % | Успех стат. Token Bucket, % | Успех стат. Sliding Window, % | p95 адапт., мс |
|-------------|-----------------|-----------------------------|-------------------------------|----------------|
| стабильная  | 100.00          | 100.00                      | 100.00                        | 8.874          |
| пуассон.    | 91.41           | 74.50                       | 19.86                         | 8.144          |
| всплеск     | 96.23           | 79.41                       | 69.13                         | 8.610          |
| DDoS        | 74.48           | 46.60                       | 5.31                          | 9.451          |
| восстановл. | 100.00          | 100.00                      | 77.28                         | 3.914          |
| Среднее     | 92.42           | 80.10                       | 54.32                         | 7.799          |

Таблица 12 показывает основной результат оптимизации: в смешанном сценарии адаптивный режим превосходит лучший статический вариант по средней доступности.

Особенно заметен эффект на **неравномерной** и **скачкообразной** нагрузке, где адаптивный режим сохраняет 91.41% и 96.23% успешных запросов соответственно.

Это важно для пользовательского API, поскольку нерегулярные локальные всплески часто являются легитимным поведением, а не атакой.

При этом в DDoS-фазе adaptive также допускает больше запросов, чем оба статических режима.

В пределах этого теста результат можно трактовать как сохранение доступности при отсутствии ошибок целевого сервиса, но не как доказательство максимальной фильтрации атакующего трафика.

Если цель состоит именно в максимальном подавлении вредоносного потока, целевую функцию адаптивного селектора необходимо менять в сторону более жёсткого профиля отклонений.

Среднее число переключений в финальном смешанном сценарии составило 1.33 на прогон.

Типовая последовательность начиналась со Sliding Window и переходила к Token Bucket примерно на 39-й секунде. В части прогонов затем происходил возврат к Sliding Window примерно на 150-й секунде.

Повторных циклов Sliding Window, Token Bucket, Sliding Window, Token Bucket внутри одного короткого смешанного прогона не было. Это означает, что на данном профиле система не переключала алгоритмы туда-сюда без необходимости.

Отдельно выполнен длительный прогон DDoS с восстановлением длительностью 720 секунд DDoS и 120 секунд на восстановление [52].

Его цель — проверить, не выходит ли адаптивный контур из **token** слишком рано при длительной атаке.

Результаты сведены в таблице 13.

Таблица 13 – Длительный прогон DDoS с восстановлением для финального адаптивного режима (24.04.2026).

| Метрика                       | Значение                                                                  |
|-------------------------------|---------------------------------------------------------------------------|
| Общая успешность              | 97.99%                                                                    |
| Успешность в DDoS             | 97.93%                                                                    |
| Успешность в восстановлении   | 100.00%                                                                   |
| Средневзвешенная задержка     | 3.917 мс                                                                  |
| Средневзвешенная p95-задержка | 9.351 мс                                                                  |
| Число переключений            | 2                                                                         |
| Последовательность режимов    | t=0 c: Sliding Window;<br>t=9 c: Token Bucket;<br>t=787 c: Sliding Window |

Длительный прогон подтверждает, что доработка защитной логики устранила прежнюю проблему раннего выхода из Token Bucket.

Контур входит в Token Bucket в начале DDoS-фазы, удерживает его почти всю атаку и выходит обратно только после восстановления.

При этом сохраняется важный компромисс: финальный профиль для смешанной нагрузки лучше работает как универсальный режим, но имеет больший p95 на чистом **DDoS** -> **восстановление**, чем более консервативный шаблон от 20.04.2026 (6.966 мс).

Этот результат показывает границу решения. Адаптивный режим лучше подстраивается под смешанную нагрузку, но всё равно приходится выбирать баланс между доступностью, жёсткой фильтрацией и задержкой.

Фазовое распределение reject для финального mixed-сценария показано на рисунке 9.

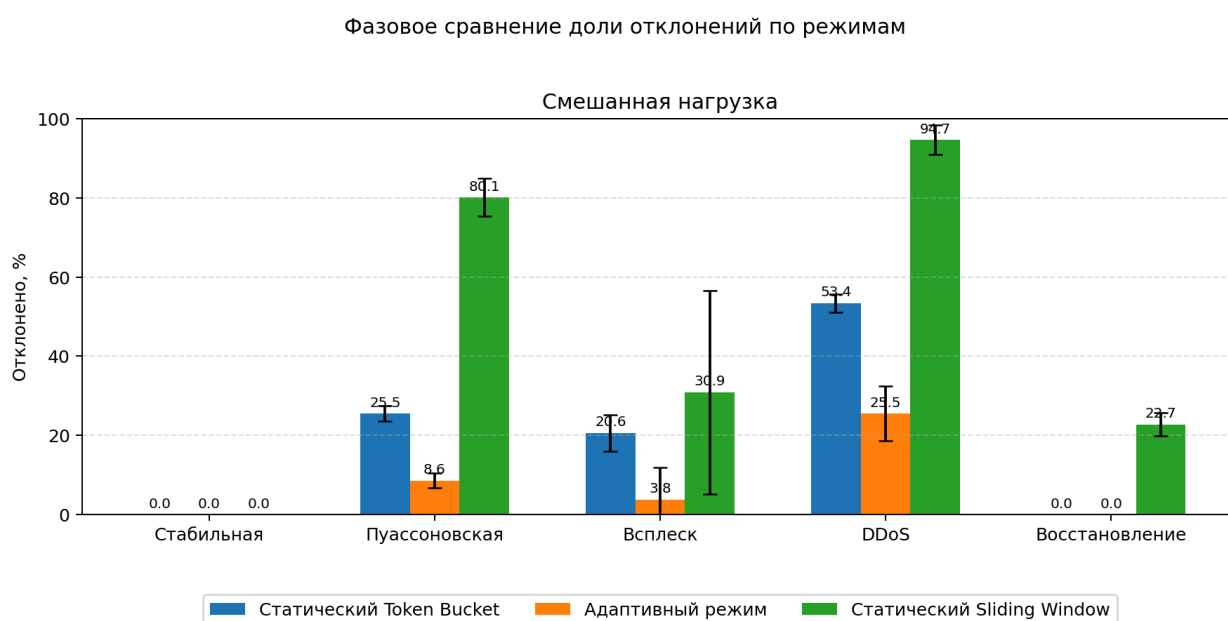


Рисунок 9 – Фазовое сравнение доли отклонений в финальном адаптивном эксперименте

Таймлайн переключений адаптивного контура для финального mixed-сценария приведён на рисунке 10.



Рисунок 10 – Диаграмма переключений адаптивного контура

Таймлайн показывает, что итоговая конфигурация воспроизводит ожидаемую последовательность переключений.

Контур выполняет ограниченное число фактических переходов и не входит в цикл постоянной смены алгоритмов.

Это важнее самого факта наличия внешнего модуля: переключение фиксируется метриками и повторяется между прогонами.

В такой архитектуре модуль управления не заменяет лимитер, а выбирает режим по наблюдаемой телеметрии.

### **Сопоставление с реальными практиками API-провайдеров.**

У крупных платформ (AWS API Gateway, Azure API Management, Cloudflare, GitHub) лимитирование тоже строится как набор управляемых правил с HTTP ответом 429 и заранее заданными квотами. Эти правила обычно не являются абсолютно точным счётчиком для каждого запроса, потому что работают в распределённой инфраструктуре [3] [4] [15] [16].

На этом фоне результат проекта соответствует ожиданиям: адаптивный контур сохраняет доступность на смешанном профиле, меняет параметры Token Bucket при перегрузке и возвращается после устойчивого восстановления.

В прототипе внешний аналитический компонент подключён как слой рекомендаций.

На вход поступают временные ряды и агрегирующие значения (задержка, скорость поступления запросов, доля отклонений, динамика

ошибок), на выход — рекомендации по параметрам лимитера и выбору режима.

Основной принцип сохраняется. Решение о пропуске или отклонении запроса остаётся в основном сервисе.

Это обеспечивает предсказуемость и отказоустойчивость даже при недоступности аналитического модуля.

Prophet используется как готовый инструмент прогнозирования временных рядов [6].

Здесь проверяется архитектурная гипотеза того, что прогноз и эвристики по телеметрии могут заранее переводить лимитер между режимами.

Финальные фазовые эксперименты подтвердили это на mixed-сценарии: настроенный селектор и автоматическая настройка параметров дали результат выше обоих статических режимов.

Итоговая оценка выполнена по трём прикладным критериям:

1. Стабильность (сохранение работоспособности целевого сервиса и отсутствие 5xx ошибок).
2. Скорость (влияние лимитера на задержку проксирования).
3. Надёжность в двух режимах (доступность для штатного трафика и фильтрация в DDoS).

В проведённых прогонах показатель стабильности оставался высоким для всех алгоритмов (ошибки **loadgen** — 0.00%), что подтверждает базовую корректность реализации [46].

Ключевые различия проявились в балансе между доступностью штатного трафика, жёсткостью защиты и задержкой.

По скорости в статической матрице лидирует Sliding Window (минимальная средняя задержка), по доступности штатного трафика близкие результаты показывают Token Bucket и Fixed Window, по жёсткости DDoS-фильтрации — Sliding Window [47].

Статический выбор одного алгоритма обоснован только при узком и стабильном профиле нагрузки.

Финальный эксперимент подтверждает этот вывод: при смешанной последовательности нагрузок адаптивный слой достигает большей средней доступности, чем **static\_token** и **static\_sliding**, за счёт управляемого выбора между Sliding Window и параметризованным Token Bucket [51].

С учётом результатов рекомендуется внедрять адаптивный режим постепенно:

1. По умолчанию запускать лимитер в безопасном режиме с **ADAPTIVE\_APPLY\_RECOMMENDATIONS=false** и наблюдением за рекомендациями.

2. Для mixed/universal нагрузки включать adaptive candidate с **TOKEN\_TUNER\_ENABLED=true**, noisy entry boost и DDoS exit guard.

3. Для сценариев, где главным KPI является минимальный p95 на длинном DDoS, использовать более консервативный эталон по задержке и не переносить профиль для смешанной нагрузки без проверки.

4. Автоматический возврат к базовому режиму выполнять только после устойчивого восстановления, а не по одиночному окну с улучшенными метриками.

Практический смысл результата — уменьшить долю лишних отказов в обычной нагрузке и усилить защитный режим при переходе к аномальному трафику.

На смешанном профиле это видно по росту усреднённой успешности адаптивного контура относительно обоих статических режимов [51].

Именно это связывает технический результат с прикладной пользой: лимитер сохраняет работающий API в неблагоприятном режиме, а не только снижает число запросов.

### **Выводы по главе 3.**

1. На локальных данных проекта (**repeats=10**) **Sliding Window** показал **лучшую скорость** (средняя задержка  $1.436 \pm 0.097$  мс), **Token Bucket** и **Fixed**



**Window — близкую доступность** для штатного трафика ( $94.87 \pm 1.76\%$  и  $94.57 \pm 1.89\%$ ), а **Sliding Window — самую жёсткую фильтрацию в DDoS** ( $42.59 \pm 6.72\%$ ) [47].

2. Эксперимент подтверждает отсутствие универсального статического лидера: выбор алгоритма должен делаться по целевой функции (скорость, доступность штатного трафика или усиленная DDoS фильтрация).

3. Ранняя фазовая серия **сценарий всплеска с восстановлением и сценарий DDoS с восстановлением** подтвердила работоспособность адаптивных переключений, но выявила чрезмерно консервативный выход из защитного режима [48] [49] [50].

4. Финальная mixed-серия **смешанный фазовый сценарий** показала, что доработанный адаптивный контур достигает 92.42% средней успешности против 80.10% у **static\_token** и 54.32% у **static\_sliding** [51].

5. Длительный прогон DDoS с восстановлением подтвердил устранение раннего выхода из Token Bucket: последовательность **0 с — Sliding Window, 9 с — Token Bucket, 787 с — Sliding Window, успешность в DDoS = 97.93%, успешность в восстановлении = 100.00%, число переключений = 2** [52].

6. Внешний аналитический компонент следует использовать как модуль выбора режима и настройки параметров, а не как замену базовых алгоритмов. Размер выигрыша зависит от цели эксперимента и профиля нагрузки.

Также стоит отметить, что в финальном адаптивном профиле Sliding Window используется как стартовый и стабилизирующий режим при спокойной нагрузке, а Token Bucket становится основным рабочим режимом при неравномерном, пиковом и аномальном трафике. Это показывает, что дальнейшее развитие прототипа может идти в сторону не переключения между разными алгоритмами, а настройки нескольких профилей Token Bucket под разные состояния нагрузки. Такой вариант упростил бы логику контура и позволил бы отдельно настраивать мягкий режим для штатного трафика и более строгий режим для защитных сценариев.

## ЗАКЛЮЧЕНИЕ

В работе исследован конкретный вопрос: какой режим ограничения запросов лучше ведёт себя на реализованном стенде RPS-limiter при штатной, перегруженной и аномальной нагрузке.

Для этого собраны воспроизводимые серии прогонов, подготовлены сценарии нагрузки, сведены метрики и проверен адаптивный контур управления между Token Bucket и Sliding Window.

Такой порядок работы позволил опереться на конкретные измерения, а не только на декларативное сравнение свойств алгоритмов.

Сравнение велось в двух режимах:

1. Штатная эксплуатация — доступность для легитимного трафика.
2. Аномальная нагрузка — стабилизация сервиса и отклонение избыточного потока.

Это разделение важно, так как один и тот же алгоритм может хорошо фильтровать DDoS, но хуже пропускать обычный трафик.

Основные результаты исследования:

1. Собран и задокументирован экспериментальный стенд: генератор нагрузки, сервис лимитирования, целевой сервис, Redis и Prometheus/Grafana.
2. Для статической матрицы проведены серии прогонов по шести сценариям нагрузки с повторениями и интервальной оценкой CI95.
3. По этим данным подтверждено, что в сценариях низкой постоянной и синусоидальной нагрузки различия по доступности отсутствуют, а в пуассоновском сценарии преимущество по сохранению легитимного трафика получает Token Bucket.
4. Для высокой постоянной нагрузки и DDoS выявлены разные компромиссы: Fixed Window мягче к клиенту, а Sliding Window обеспечивает наиболее жёсткую фильтрацию.
5. Добавлен и проверен адаптивный контур, который выбирает между Token Bucket и Sliding Window по телеметрии и рекомендациям внешнего модуля.

6. После ранних фазовых прогонов в контур внесены обнаружение шумной перегрузки, автоматическая настройка параметров Token Bucket и защитное условие выхода из DDoS-режима.

7. В смешанном фазовом сценарии финальная версия адаптивного контура достигает 92.42% средней успешности против 80.10% у static\_token и 54.32% у static\_sliding [51].

8. В длительном прогоне DDoS -> восстановление контур сохранил 97.99% общей успешности и выполнил только два переключения, что подтвердило исправление прежнего раннего выхода из защитного режима [52].

Дополнительно по результатам стенда сформулирован порядок внедрения лимитера:

1. Старт с базового режима.
2. Наблюдение за метриками и оценка разброса результатов на повторях.
3. Включение адаптивного контура с защитными ограничениями и безопасным откатом.
4. Регулярная ревизия политики регулирования при изменении контрактов API и профиля нагрузки.

Эти шаги вытекают из результатов стенда и задают порядок безопасного включения адаптивного контура.

Цель работы можно считать достигнутой: выбранная архитектура реализована, измерена и проверена на сериях воспроизводимых экспериментов.

Результаты показывают, при каких профилях нагрузки лучше подходят Token Bucket, Sliding Window или адаптивный режим.

Адаптивный контур полезен как управляемый селектор режима и параметров, но не заменяет детерминированный лимитер.

Для применения на практике из этого можно выделить два правила:

1. Не оценивать защиту без единовременной проверки доступности штатного трафика.

2. Включать адаптивный контур только с наблюдением, защитной логикой и процедурой отката.

Ограничения исследования:

1. Эксперимент выполнен на контролируемом стенде и ограниченном наборе сценариев.

2. Не проводилось сравнение разных ML-моделей между собой.

3. Часть факторов, таких как многорегиональность и сложные сетевые аномалии, не моделировалась в полном объёме.

4. DDoS-сценарий стенда не разделяет вредоносных и легитимных клиентов по ключам, поэтому высокую успешность в атакующей фазе следует трактовать как сохранение доступности стенда, а не как доказательство точной фильтрации атакующего источника.

5. Финальный профиль для смешанной нагрузки повышает универсальность, но не является лучшим одновременно по всем метрикам.

Перспективы дальнейшей работы:

1. Расширение сценариев тестирования на многорегиональную конфигурацию и искусственное внесение сетевых разделений.

2. Добавление многоуровневой политики лимитов (глобальный уровень, клиент, конечная точка, метод).

3. Интеграция адаптивной политики с бизнес-приоритетами запросов.

4. Формализация SLO/SLA для режима адаптивного ограничения и автоматических механизмов отката.

5. Подбор параметров адаптивного селектора на наборе повторяемых сценариев, включая отдельную оптимизацию под смешанный трафик и под жёсткую DDoS-фильтрацию.

6. Расширение онлайн-настройки: управление не только **fillRate** и **capacity**, но и профилями по клиенту и маршруту, чтобы различать штатный всплеск и атакующий поток.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Cloudflare Radar. DDoS threat report for 2025 Q1 [Электронный ресурс]. URL: <https://radar.cloudflare.com/reports/ddos-2025-q1> (дата обращения: 28.02.2026).
2. Cloudflare Radar. DDoS threat report for 2025 Q4 [Электронный ресурс]. URL: <https://radar.cloudflare.com/reports/ddos-2025-q4> (дата обращения: 28.02.2026).
3. Amazon API Gateway. Throttle requests to your REST APIs for better throughput in API Gateway [Электронный ресурс]. URL: <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html> (дата обращения: 28.02.2026).
4. Azure API Management policy reference: rate-limit-by-key [Электронный ресурс]. URL: <https://learn.microsoft.com/en-us/azure/api-management/rate-limit-by-key-policy> (дата обращения: 28.02.2026).
5. Redis Docs. Redis cluster specification [Электронный ресурс]. URL: [https://redis.io/docs/latest/operate/oss\\_and\\_stack/reference/cluster-spec/](https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/) (дата обращения: 28.02.2026).
6. Prophet (facebook/prophet). Time series forecasting library [Электронный ресурс]. URL: <https://github.com/facebook/prophet> (дата обращения: 28.02.2026).
7. Google SRE Book. Addressing Cascading Failures [Электронный ресурс]. URL: <https://sre.google/sre-book/addressing-cascading-failures/> (дата обращения: 28.02.2026).
8. Cloudflare Security Blog. Memcrashed: Major amplification attacks from UDP port 11211 [Электронный ресурс]. URL: <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211/> (дата обращения: 28.02.2026).

9. NVD. CVE-2018-1000115 [Электронный ресурс]. URL: <https://nvd.nist.gov/vuln/detail/CVE-2018-1000115> (дата обращения: 28.02.2026).
10. ISO/IEC 27001:2022 Information security management systems — Requirements [Электронный ресурс]. URL: <https://www.iso.org/standard/27001> (дата обращения: 28.02.2026).
11. NIST SP 800-53 Rev.5. Security and Privacy Controls for Information Systems and Organizations [Электронный ресурс]. URL: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final> (дата обращения: 28.02.2026).
12. Nottingham M., Fielding R. Additional HTTP Status Codes (RFC 6585). IETF, 2012 [Электронный ресурс]. URL: <https://www.rfc-editor.org/rfc/rfc6585> (дата обращения: 28.02.2026).
13. AWS Well-Architected Framework. REL05-BP02 Throttle requests [Электронный ресурс]. URL: [https://docs.aws.amazon.com/wellarchitected/latest/framework/rel\\_mitigate\\_interaction\\_failure\\_throttle\\_requests.html](https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_mitigate_interaction_failure_throttle_requests.html) (дата обращения: 28.02.2026).
14. Amazon API Gateway. Throttle requests to your HTTP APIs for better throughput in API Gateway [Электронный ресурс]. URL: <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-throttling.html> (дата обращения: 28.02.2026).
15. Cloudflare Developers. Rate limits [Электронный ресурс]. URL: <https://developers.cloudflare.com/fundamentals/api/reference/limits/> (дата обращения: 28.02.2026).
16. GitHub Docs. Rate limits for the REST API [Электронный ресурс]. URL: <https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api> (дата обращения: 28.02.2026).
17. OpenAPI Initiative. OpenAPI Specification [Электронный ресурс]. URL: <https://spec.openapis.org/oas/latest.html> (дата обращения: 28.02.2026).

18. Hardt D. The OAuth 2.0 Authorization Framework (RFC 6749). IETF, 2012 [Электронный ресурс]. URL: <https://www.rfc-editor.org/rfc/rfc6749> (дата обращения: 28.02.2026).
19. OWASP. OWASP API Security Top 10 — 2023 [Электронный ресурс]. URL: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (дата обращения: 28.02.2026).
20. Spring Cloud Gateway. RequestRateLimiter GatewayFilter Factory [Электронный ресурс]. URL: <https://docs.spring.io/spring-cloud-gateway/reference/spring-cloud-gateway-server-webflux/gatewayfilter-factories/requestratelimiter-factory.html> (дата обращения: 28.02.2026).
21. NGINX. Module ngx\_http\_limit\_req\_module [Электронный ресурс]. URL: [https://nginx.org/en/docs/http/ngx\\_http\\_limit\\_req\\_module.html](https://nginx.org/en/docs/http/ngx_http_limit_req_module.html) (дата обращения: 28.02.2026).
22. Bucket4j/bucket4j. Java rate limiting library based on token-bucket algorithm [Электронный ресурс]. URL: <https://github.com/bucket4j/bucket4j> (дата обращения: 28.02.2026).
23. Resilience4j. RateLimiter [Электронный ресурс]. URL: <https://resilience4j.readme.io/docs/ratelimiter> (дата обращения: 28.02.2026).
24. Redis Docs. INCR command (including rate limiter patterns) [Электронный ресурс]. URL: <https://redis.io/docs/latest/commands/incr/> (дата обращения: 28.02.2026).
25. Prometheus. Overview [Электронный ресурс]. URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения: 28.02.2026).
26. Ширяев Е.М., Назаров А.С., Кучеров Н.Н., Бабенко М.Г. Аналитический обзор конфиденциального искусственного интеллекта: методы и алгоритмы реализации в облачных вычислениях // Программирование. 2024. № 4. С. 27-40. DOI: 10.31857/S0132347424040036.

27. Славин О.А. Optimization of software performance for classification and linking of administrative documents // Программирование. 2024. № 6. С. 48-58. DOI: 10.31857/S0132347424060057.

28. Бенерджи К.Г., Гупта М.К. Компромиссное соотношение между стоимостью хранения и восстановления для неоднородных распределенных систем хранения данных // Проблемы передачи информации. 2021. Т. 57. № 1. С. 40-63. DOI: 10.31857/S0555292321010022.

29. Колногоров А.В. Пуассоновский двурукий бандит: новый подход // Проблемы передачи информации. 2022. Т. 58. № 2. С. 66-91. DOI: 10.31857/S0555292322020065.

30. Лихтциндер Б.Я., Привалов А.Ю., Моисеев В.И. Неординарные пуассоновские модели трафика мультисервисных сетей // Проблемы передачи информации. 2023. Т. 59. № 1. С. 71-79. DOI: 10.31857/S0555292323010060.

31. Гао В., Башконуш Х.М., Каттани К. Эффективность передачи данных при атаках с точки зрения варианта изолированной жесткости // Проблемы передачи информации. 2023. Т. 59. № 2. С. 83-101. DOI: 10.31857/S0555292323020067.

32. Миллер А.В. Синхронизация времени в спутниковом квантовом распределении ключей // Проблемы передачи информации. 2023. Т. 59. № 4. С. 13-27. DOI: 10.31857/S0555292323040022.

33. Лихтциндер Б.Я., Привалов А.Ю. Обобщение формул для моментов очереди при неординарном пуассоновском потоке для очередей пакетов в системах телекоммуникаций // Проблемы передачи информации. 2023. Т. 59. № 4. С. 32-37. DOI: 10.31857/S0555292323040046.

34. Карамышев А.Ю., Порай Е.Д., Хоров Е.М. Оценка емкости системы сверхнадежной связи с низкими задержками с помощью аппроксимаций для многосерверных систем массового обслуживания G/G/s // Проблемы передачи информации. 2024. Т. 60. № 2. С. 36-52. DOI: 10.31857/S0555292324020049.



35. Банков Д.В., Ляхов А.И., Степанова Е.А., Хоров Е.М. Анализ влияния механизма restricted target wake time на пропускную способность сети Wi-Fi // Проблемы передачи информации. 2024. Т. 60. № 3. С. 59-75. DOI: 10.31857/S0555292324030070.

36. Федорищева А.А., Банков Д.В., Ляхов А.И., Хоров Е.М. Математическое моделирование сети LoRaWAN при совместном обслуживании подтверждаемого и неподтверждаемого типов трафика // Проблемы передачи информации. 2024. Т. 60. № 4. С. 44-57. DOI: 10.31857/S055529232404003X.

37. Karpowicz M.P. Adaptive tuning of network traffic policing mechanisms for DDoS attack mitigation systems // European Journal of Control. 2021. Vol. 61. P. 101-118. DOI: 10.1016/j.ejcon.2021.07.001.

38. Li C., Liu J., Lu C., Guerin R., Gill C.D. Impact of Distributed Rate Limiting on Load Distribution in a Latency-sensitive Messaging Service // 2021 IEEE 14th International Conference on Cloud Computing (CLOUD). 2021. P. 367-377. DOI: 10.1109/CLOUD53861.2021.00050.

39. Eliyan L.F., Di Pietro R. DoS and DDoS attacks in Software Defined Networks: A survey of existing solutions and research challenges // Future Generation Computer Systems. 2021. Vol. 122. P. 149-171. DOI: 10.1016/j.future.2021.03.011.

40. El Kamel A., Eltaief H., Youssef H. On-the-fly (D)DoS attack mitigation in SDN using Deep Neural Network-based rate limiting // Computer Communications. 2022. Vol. 182. P. 153-169. DOI: 10.1016/j.comcom.2021.11.003.

41. Patidar A., Somani G. Serving while attacked: DDoS attack effect minimization using page separation and container allocation strategy // Journal of Information Security and Applications. 2021. Vol. 59. Article 102818. DOI: 10.1016/j.jisa.2021.102818.

42. Karnani S., Shakya H.K. Mitigation strategies for distributed denial of service (DDoS) in SDN: A survey and taxonomy // Information Security Journal:

A Global Perspective. 2023. Vol. 32. No. 6. P. 444-468. DOI: 10.1080/19393555.2022.2111004.

43. Al-Dunainawi Y., Al-Kaseem B.R., Al-Raweshidy H.S. Optimized Artificial Intelligence Model for DDoS Detection in SDN Environment // IEEE Access. 2023. Vol. 11. P. 106733-106748. DOI: 10.1109/ACCESS.2023.3319214.

44. Novaes M.P., Carvalho L.F., Lloret J., Proenca M.L. Jr. Adversarial Deep Learning approach detection and defense against DDoS attacks in SDN environments // Future Generation Computer Systems. 2021. Vol. 125. P. 156-167. DOI: 10.1016/j.future.2021.06.047.

45. RPS-limiter project. Benchmark matrix with repeated runs (static mode, 2026-03-01, duration 12s, repeats 10) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 01.03.2026).

46. RPS-limiter project. Raw benchmark results with repeats (static mode, 2026-03-01) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 01.03.2026).

47. RPS-limiter project. Aggregated benchmark overview with CI95 (static mode, 2026-03-01) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 01.03.2026).

48. RPS-limiter project. Adaptive phase benchmark raw results (**сценарий всплеска с восстановлением, сценарий DDoS с восстановлением**, 20.03.2026, repeats 6) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 20.03.2026).

49. RPS-limiter project. Adaptive phase benchmark aggregated summary with CI95 (20.03.2026) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 20.03.2026).

50. RPS-limiter project. Adaptive phase benchmark switch summary and timeline (20.03.2026) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 20.03.2026).

51. RPS-limiter project. Adaptive universal mixed benchmark (**смешанный фазовый сценарий**, 24.04.2026, repeats 3) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 24.04.2026).

52. RPS-limiter project. Adaptive длительный прогон benchmark (**DDoS -> восстановление**, 24.04.2026) [Электронный ресурс]. URL: <https://github.com/Islam-Y/RPS-limiter/tree/main/services/monitoring/benchmarks> (дата обращения: 24.04.2026).

## **СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА**

Рисунок 1 – Перераспределение нагрузки при отказе экземпляра сервиса

Рисунок 2 – Сравнение подходов Token Bucket и Leaky Bucket

Рисунок 3 – Архитектура API-шлюза с контуром ограничения запросов

Рисунок 4 – Структура экспериментальных результатов по сценариям

Рисунок 5 – Схема экспериментального стенда

Рисунок 6 – Задержка прокси по алгоритмам

Рисунок 7 – Надёжность: успешность штатного трафика и отклонения при DDoS

Рисунок 8 – Карта отклонений по сценариям и алгоритмам

Рисунок 9 – Фазовое сравнение доли отклонений в финальном адаптивном эксперименте

Рисунок 10 – Диаграмма переключений адаптивного контура